

NEXTGENMMA

Propuesta de Proyecto DAM
DESARROLLO DE UNA APLICACIÓN DE GESTIÓN

Diego Gil Sanz
25209587T

Desarrollo de aplicaciones Multiplataforma
Curso académico 2023-25

Tutor de Proyecto
José Manuel Guallar

Contenido

Propuesta de Proyecto DAM	1
Diego Gil Sanz.....	1
Desarrollo de aplicaciones Multiplataforma	1
Tutor de Proyecto	1
1. Justificación y planificación del proyecto.....	3
1.1.- Descripción del proyecto.....	3
1.2.- Justificación de la tecnología utilizada	3
1.3.- Planificación Gantt	4
Primera semana.....	4
1.4.- Planificación Scrum.....	7
1.5.- Prototipado	10
2. Plan de empresa	11
2.1.- Oportunidades de negocio y necesidades demandadas	11
2.2.- Relación de productos o servicios a prestar	12
2.3.- Constitución de la empresa.....	12
2.4.- Recursos.....	13
3. Análisis y diseño de los requisitos de la aplicación	16
3.1.- Herramientas escogidas	18
3.2.- Diagrama de casos de uso	19
3.4.- MER	21
4. Desarrollo de la aplicación	22
4.1.- Control de usuarios.....	22
4.2.- Desarrollo e implementación de la aplicación	36
MAPEOS	37
5. Pruebas e informes de la aplicación.....	55
5.1.- Estrategia de pruebas.....	55
5.2.- Pruebas realizadas	55
5.3.- Informes	56
6. Implantación y documentación	58
6.1.- Exportación y distribución de la aplicación	58
6.2.- Manual de instalación	58
6.3.- Manual de usuario	58
6.4.- Repositorio de la aplicación	59
6.5.- JavaDoc.....	60
7.- Bibliografía	61
8.- Anexos	62

1. Justificación y planificación del proyecto

1.1.- Descripción del proyecto

NEXTGENMMA es una aplicación que busca facilitar toda la gestión de la federación española de mma, busca facilitar las inscripciones de nuevos clubes, gimnasio y ligas.

Las federaciones no evolucionan y los peleadores si, los peleadores quieren lo mejor y se merecen lo mejor.

Nuestra aplicación aportara una solución a los tiempos de espera y el desplazamiento de los trabajadores a las oficinas, contaremos con una función de enviar solicitudes a el correo de la federación, para llevar a cabo todas las solicitudes de forma mas rápida y sin necesidad de llevar horarios de las oficinas.

Podremos eliminar, modificar y añadir los peleadores gracias a la ventana de administración.

También es capaz de mostrar diferentes anuncios/publicaciones para que los usuarios estén bien informados.

1.2.- Justificación de la tecnología utilizada

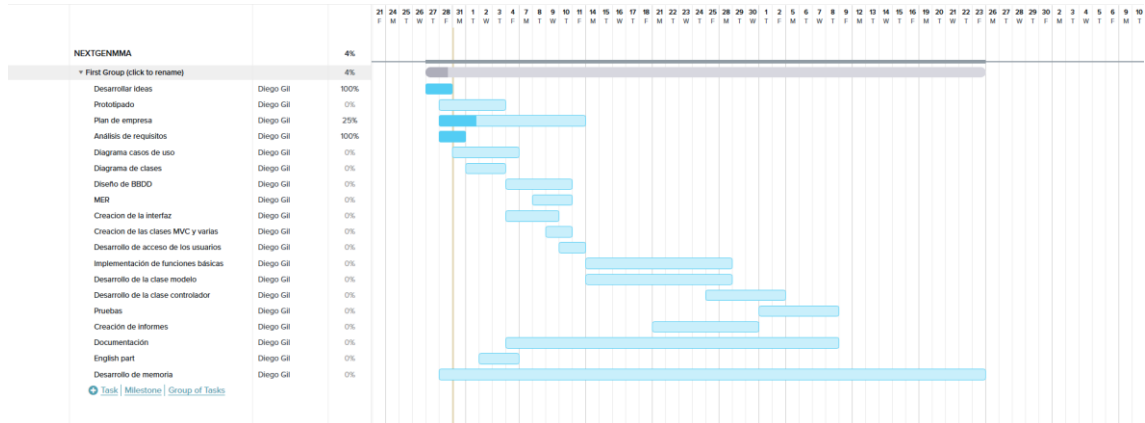
He elegido hibernate debido a su facilidad de mapeo, facilidad de gestión de datos y porque es mas sencillo crear código, me permite simplificar las consultas y operaciones, también tiene una cualidad muy importante y es que es rápida, da mas fluidez a la aplicación.

He elegido Java como lenguaje debido a que estoy acostumbrado a crear proyectos con el, tengo un control suficiente para crear programas con diversas funciones y que sea muy completo, además, es el lenguaje que mas he usado.

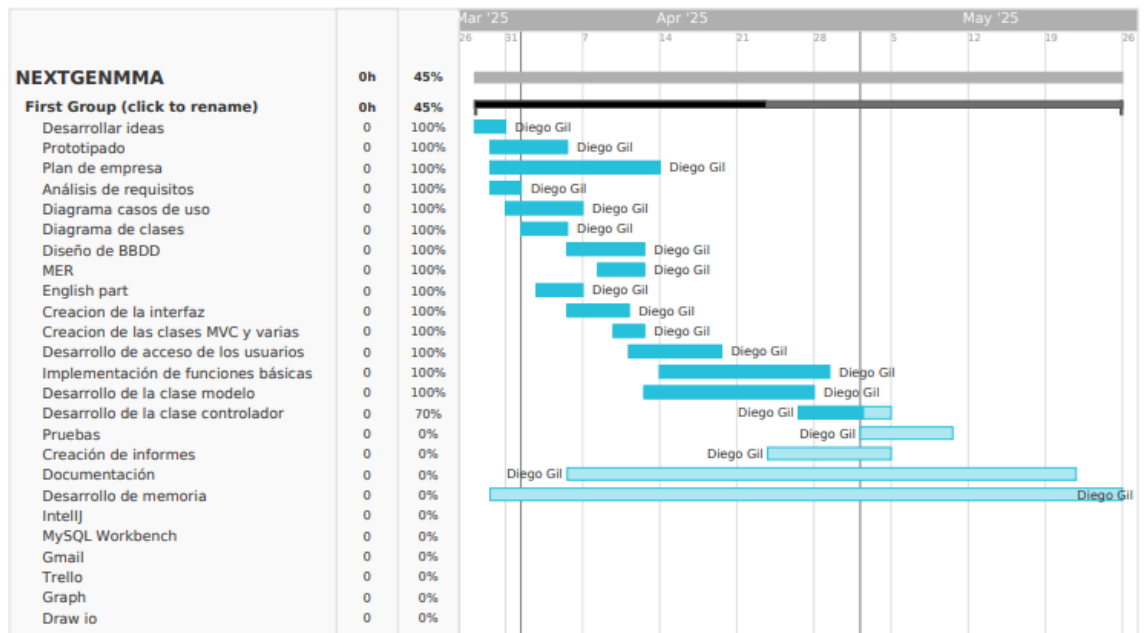
Estoy usando el entorno de desarrollo de IntelliJ debido a que la interfaz es muy comoda y me resulta mas fácil desarrollar código, además, me resulta un entorno de desarrollo muy rápido con una gran cantidad de herramientas y posibilidades

1.3.- Planificación Gantt

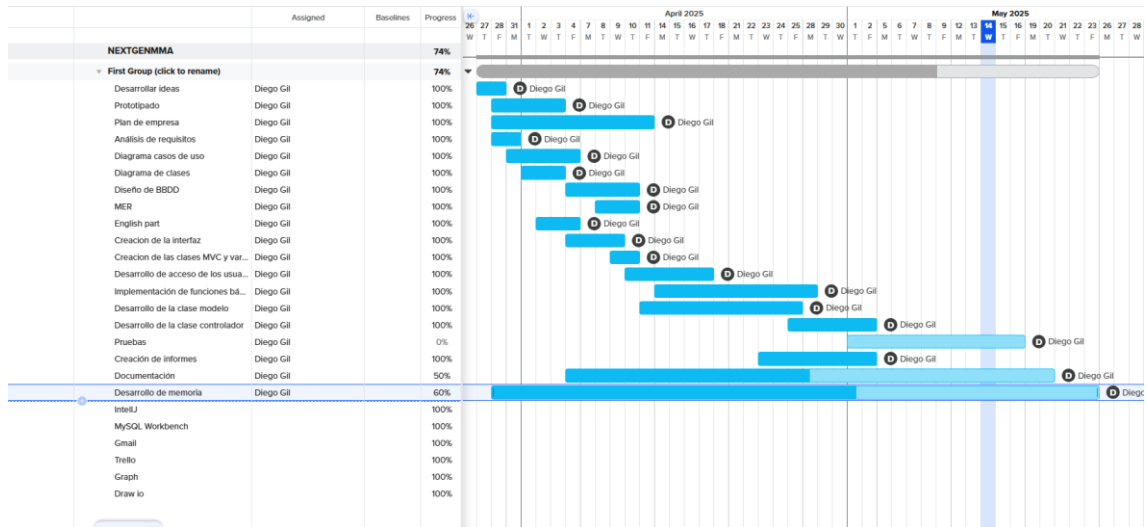
Primera semana



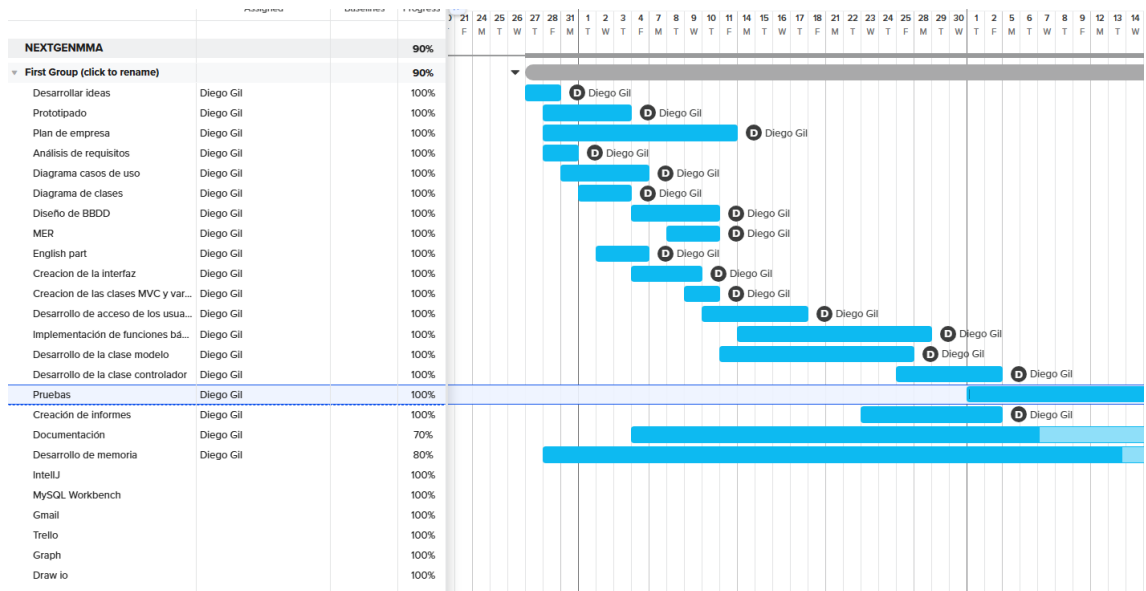
Primeras 3 semanas



Progreso de 5 semanas



Progreso 7 semanas



		4 25 26 27 28 31 1 2 3 4 7 8 9 10 11 14 15 16 17 18 21 22 23 24 25 28 29 30 1 2 5 6 7 8 9 12 13 14 15 16 19 20 21 22 23																												
		A T W T F M T W T F M T W T F M T W T F M T W T F M T W T F M T W T F																												
NEXTGENMMA		100%																												
▼ First Group (click to rename)		100%																												
Desarrollar ideas	Diego Gil	100%																												
Prototipado	Diego Gil	100%																												
Plan de empresa	Diego Gil	100%																												
Análisis de requisitos	Diego Gil	100%																												
Diagrama casos de uso	Diego Gil	100%																												
Diagrama de clases	Diego Gil	100%																												
Diseño de BBDD	Diego Gil	100%																												
MER	Diego Gil	100%																												
English part	Diego Gil	100%																												
Creacion de la interfaz	Diego Gil	100%																												
Creacion de las clases MVC y varias	Diego Gil	100%																												
Desarrollo de acceso de los usuarios	Diego Gil	100%																												
Implementación de funciones básicas	Diego Gil	100%																												
Desarrollo de la clase modelo	Diego Gil	100%																												
Desarrollo de la clase controlador	Diego Gil	100%																												
Pruebas	Diego Gil	100%																												
Creación de informes	Diego Gil	100%																												
Documentación	Diego Gil	100%																												
Desarrollo de memoria	Diego Gil	100%																												
IntelliJ		100%																												
MySQL Workbench		100%																												
Gmail		100%																												
Trello		100%																												
Graph		100%																												
Draw.io		100%																												

1.4.- Planificación Scrum

NEXTGENMMA Visible para el Espacio de trabajo Tablero Tabla Cronología

TO DO

Análisis de requisitos
🕒 30 mar - 31 mar
📊 100% 📅 Mar 31

Diagrama de clases
🕒 1 abr - 3 abr
📊 0% 📅 Apr 3

Diseño de BBDD
🕒 4 abr - 10 abr
📊 0% 📅 Apr 10

Creación de la interfaz
🕒 6 abr - 9 abr
📊 0% 📅 Apr 9

Creación de las clases MVC y vistas
🕒 9 abr - 10 abr
📊 0% 📅 Apr 10

Implementación de funciones básicas
🕒 14 abr - 28 abr
📊 0% 📅 Apr 28

Desarrollo de la clase modelo
🕒 14 abr - 28 abr
📊 0% 📅 Apr 28

Desarrollo de la clase controlador
🕒 25 abr - 2 may
📊 0% 📅 May 2

Creación de informes
🕒 21 abr - 30 abr
📊 0% 📅 Apr 30

Desarrollo de acceso de los usuarios
🕒 10 abr - 15 abr
📊 0% 📅 Apr 15

Documentación
🕒 4 abr - 8 may
📊 0% 📅 May 8

Pruebas
🕒 1 may - 8 may
📊 0% 📅 May 8

Desarrollo de memoria
🕒 30 mar - 23 may
📊 0% 📅 May 23

MER
🕒 8 abr - 10 abr
📊 0% 📅 Apr 10

Prototipado
🕒 30 mar - 3 abr
📊 0% 📅 Apr 3

Diagrama casos de uso
🕒 31 mar - 4 abr
📊 0% 📅 Apr 4

+ Añadir una tarjeta

IN PROGRESS

Plan de empresa
🕒 30 mar - 11 abr
📊 25% 📅 Apr 11

English part
🕒 2 abr - 4 abr
📊 0% 📅 Apr 4

+ Añadir una tarjeta

FINISH

✓ Desarrollar ideas
🕒 27 mar - 30 mar
📊 100% 📅 Mar 30

+ Añadir una tarjeta

PROBLEMS

+ Añadir una tarjeta

RESSOURCES

IntelliJ
📊 0%

MySQL Workbench
📊 0%

Gmail
📊 0%

Trello
📊 0%

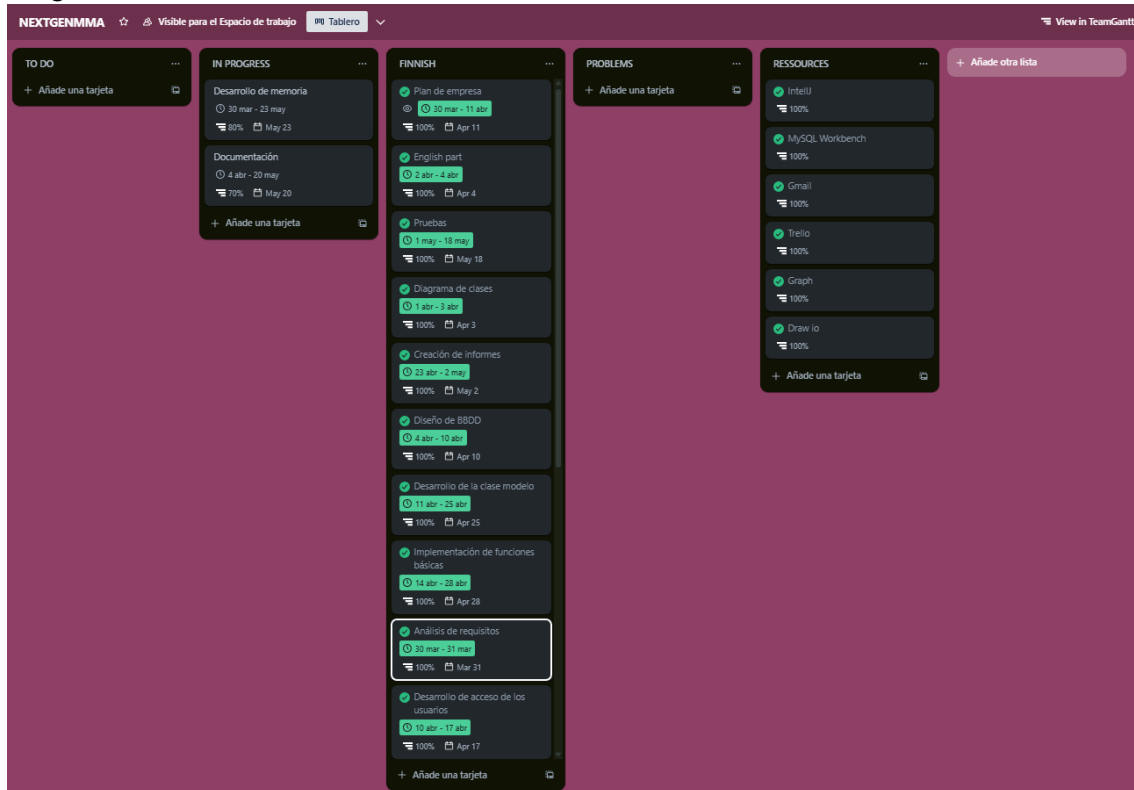
Graph
📊 0%

Draw.io
📊 0%

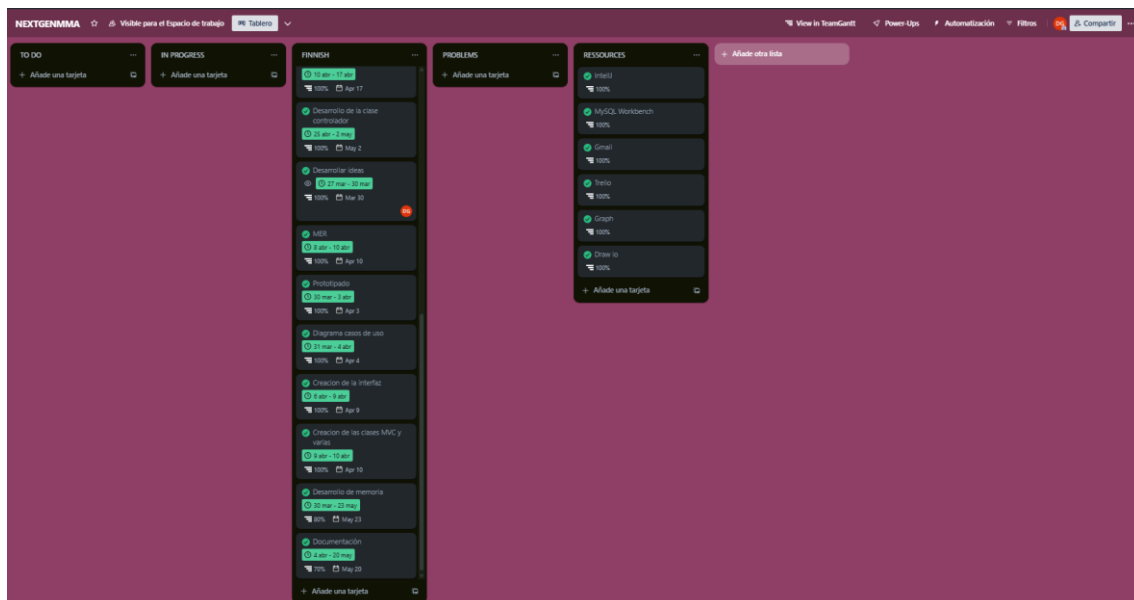
+ Añadir una tarjeta

[illegible][illegible]

Progreso 7 semanas



Progreso 9 semanas



1.5.- Prototipado

<https://www.figma.com/design/QT8KhX3sQUVjjTT8GcUW1d/TFG-MMA?node-id=1-1403&t=coS1ciNZcFbV1opu-1>

2. Plan de empresa

2.1.- Oportunidades de negocio y necesidades demandadas

Justificación de la oportunidad de negocio

Las federaciones de los múltiples deportes que se practican en España no usan sus aplicaciones y en algunos casos dejándolas descuidadas, como miembro de un club de artes marciales mixtas, he podido observar que más que nunca, en esta era informática necesitamos evolucionar a la hora de hacer tramites. Por lo que sé, no hay muchas empresas interesadas en poder rehacer algunas de las páginas web de las federaciones.

Al no tener mucha competencia, podemos aprovechar para negociar con cada federación y que logren apostar por nosotros, personas que hemos practicado estos deportes y sabemos cómo lo queremos y lo que necesitamos

Descripción de la localización de la empresa.

La localización elegida sería Zaragoza, una ciudad en la que los miembros de la empresa estamos familiarizados.

Apostaremos por el talento local, el cual está en auge, cada día son más los jóvenes que se aventuran en el mundo del desarrollo de aplicaciones y Aragón no se queda atrás. Sabemos que, en Madrid, Barcelona, Valencia, son ciudades grandes que cuentan con una gran cantidad de talento joven, pero nosotros queremos apostar por los jóvenes de Aragón para brindarles una oportunidad de elevar su carrera a un nivel superior sin necesidad de tener que salir de su tierra, además somos una empresa con valores ultra competitivos, buscamos crecer como empresa y llegar a demostrar que Aragón tiene un potencial único en el sector. No solo nos basamos en el talento joven, también nos hemos asegurado de que Zaragoza nos brinde alguna ayuda. Podremos acceder a deducciones fiscales gracias a programas que ayudan a impulsar a las pequeñas empresas, hay otro plan de ayudas que nos ayuda a reducir el IBI, nos aporta un gran porcentaje de descuento a la hora de pagar el IBI y el IAE.

Estudio de nuestro cliente y segmentación de mercado

Clientes

Nuestros clientes objetivos son las federaciones deportivas, principalmente la federación española de MMA, pero buscamos también expandirnos a diversas federaciones de los diferentes deportes que se practican en España

2.2.- Relación de productos o servicios a prestar

Producto/servicio

El producto principal que ofrecemos es una aplicación de gestión que permitirá a los administradores añadir, modificar y eliminar nuevos gimnasios, peleadores, ligas o entrenadores. Permitimos que los gimnasios tengan más facilidad para comunicarse con la federación gracias a la función de mandar un mensaje directo a soporte para tratar la solicitud. Las personas que no pertenezcan a ningún club podrán acceder a la sección de POST para estar al día de las últimas noticias de las MMA.

Precio

La aplicación principalmente será gratis para que todos los clubes y federaciones la puedan usar. Al ser una aplicación gratis los fans de las MMA podrán usarla para estar actualizados en todo momento, pero, si no eres fan, podrás descubrir el mundo de las artes marciales mixtas cómodamente y de forma totalmente gratuita.

Promoción

Ofrecemos una aplicación gratuita, no podemos hacer rebajas en nuestro precio por lo tanto las federaciones que decidan usar nuestra aplicación, les ofrecemos un curso formativo gratuito para que sus trabajadores sean totalmente eficientes.

Publicidad

NEXTGENMMA se dará a conocer a través de las redes sociales, publicando contenido sobre las novedades de las MMA y diversas curiosidades. Crearemos contenido de humor para los jóvenes y al final de cada video promocionaremos nuestra aplicación para que el público una vez entretenido, pueda descubrirnos y tener más posibilidades de llegar a ellos.

Distribución

Nuestra aplicación se podrá obtener principalmente desde nuestra página web, pero, también facilitamos a los usuarios que nos siguen en nuestras redes sociales poder obtenerla. Tendremos un enlace de descarga directa de la última versión en la biografía de nuestras RRSS para que sea más rápido y cómodo poder descubrirnos.

2.3.- Constitución de la empresa

Tipo de sociedad elegido explicando las razones

NEXTGENMMA es una sociedad limitada, ¿Por qué?, una sociedad limitada nos brinda muchas ventajas al ser una empresa pequeña y con poco capital, nos aseguramos protección gracias a la responsabilidad limitada, no tenemos que responder con el capital personal y es una gran ventaja al ser una empresa pequeña que está comenzando, minimizamos posibles daños y aportamos una imagen profesional, asegurando a los posibles inversores o clientes que no hay un gran riesgo de pérdidas.

Somos conscientes de que al principio tenemos que pagar 3500 € aproximadamente para cubrir todos los gastos iniciales y de mantenimiento de la sociedad, pero tener la seguridad de que no tenemos que poner el capital personal es lo importante.

Otro punto muy importante es la escalabilidad que permite, empezaremos siendo pocos socios, pero en el futuro nos gustaría ser más, para crecer y desarrollarnos, la sociedad limitada nos permite crecer.

Por ultimo y no menos importante, los impuestos, podremos aprovechar las deducciones y nos beneficiaremos de un tipo impositivo reducido los primeros años.

Nombra la relación de trámites mercantiles, fiscales, laborales y locales que debe realizar la empresa

❖ TRAMITES MERCANTILES

- **Certificación negativa del nombre**
 - Sirve para comprobar que el nombre no está en uso y está disponible
 - Se solicita en el Registro Mercantil Central
 - Costo de 16€
- **Redacción de Estatutos Sociales**
 - Es el Documento que define la actividad de la empresa, también contiene el reparto de las participaciones y cómo funciona
- **Deposito del capital social**
 - Al ser una sociedad limitada tenemos que tener ingresado mínimo 3000€ en una cuenta de banco a nombre de la empresa
- **Escritura de constitución ante el notario**
 - Todos los miembros deben firmar en la escritura publica
 - Costo= 150-300€
- **Inscripción en el Registro Mercantil**
 - Da personalidad jurídica a la empresa
 - Costo=200-300

❖ TRAMITES FISCALES

- **Obtención del NIF provisional**
 - Se solicita en la Agencia Tributaria (hacienda)
 - Hay que tenerlo antes de registrar la sociedad
- **Alta en el censo de empresarios**
 - Necesitamos estar de alta para declarar los impuestos
- **Obtención del NIF definitivo**
 - Una vez registrados en el registro mercantil tenemos que solicitarlo
- **Declaración del impuesto de Actividades Económicas (PROXIMAMENTE)**
 - Si la empresa llega a generar más de 1 millón de euros anuales tendríamos que declararlo, pero de momento al ser nueva y sin generar mucho estamos exentos
- **Alta en el impuesto de sociedades**
 - La empresa tributa por el Impuesto de Sociedades

➤ **Declaración trimestral y anual de impuestos**

- IVA
- Retenciones de IRPF
- Impuesto de sociedades

❖ **TRAMITES LABORALES**

➤ **Inscripción de la empresa en la Seguridad Social**

- **NEXTGENMMA** tendrá contratados varios trabajadores por lo tanto tiene que estar inscrita

➤ **Alta del administrador en el RETA**

- Al principio al ser el único administrador de la sociedad, tendré que estar dado de alta, en un futuro si se divide la administración podría ser que no hiciera falta

Alta de trabajadores en la Seguridad Social

- Les daremos de alta en la RED de la tesorería general de la seguridad social
- **Prevención de riesgos laborales**
 - Al tener trabajadores estamos obligados a tener un servicio de prevención de riesgos laborales

Por el momento estos son los tramites que tenemos que realizar, no necesitamos Trámites Locales debido a que cada trabajador realizara su actividad desde su domicilio, estos nos ahorran una gran cantidad de dinero, en un futuro cuando decidamos montar oficinas sí que tendríamos que pagar las **tasas municipales**, **Registrarnos en el padrón municipal** y obtener la **Licencia de actividad y apertura**.

2.4.- Recursos

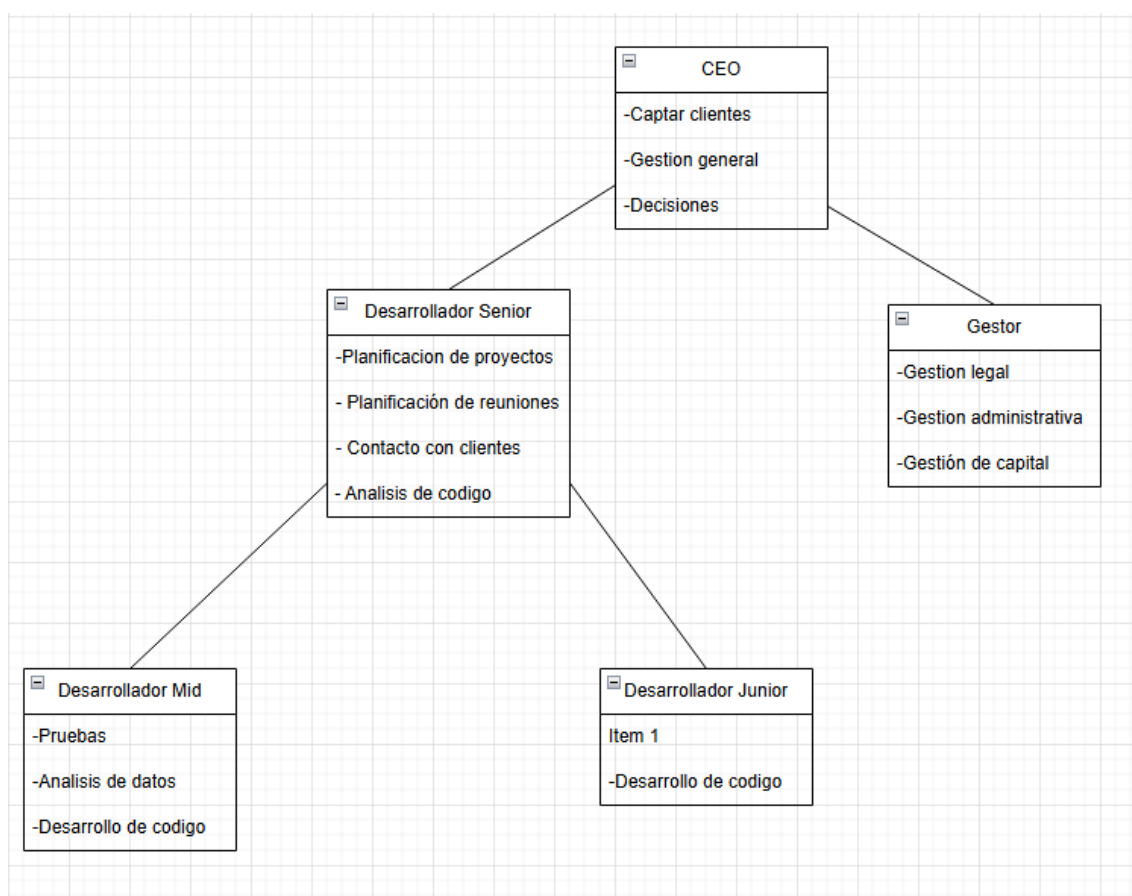
Recursos Materiales/Técnicos

ELEMENTO	MENSUAL	ANUAL
Licencias IntelliJ (4 licencias)	83€	996€
GitHub team (4 cuentas)	16€	192€
GitHub copilot (4 cuentas)	40€	480€
Servidor en la nube	8€	96€
Licencias Trello (4 cuentas)	40€	480€
Total, mensual	187€	
Total, anual		2244€

ELEMENTO	COSTE
Ordenador sobremesa (4 unidades)	4800€
Monitor (4 unidades)	800€
Periféricos (4 unidades)	280€
Mesa (4 unidades)	800€
Sillas (4 unidades)	400€
TOTAL	7080€

Recursos Humanos

Concepto	MENSUAL	ANUAL
Desarrollador Jr. (2 desarrolladores)	3640€	43680€
Desarrollador Mid	2340€	28080€
Desarrollador Sr	3250€	39.000€
Gestor	300€	3600€
TOTAL	9530€	114360€



Recursos Económicos**INVERSION INICIAL**

CONCEPTO	COSTE
Recursos humanos	9530€
Recursos físicos	7080€
Recursos humanos	114360€
Fondos SL	3000€
Notaria	200€
Obtención del NIF	20-50€
INVERSION INICIAL	134.200€

3. Análisis y diseño de los requisitos de la aplicación

3.1.- Herramientas escogidas

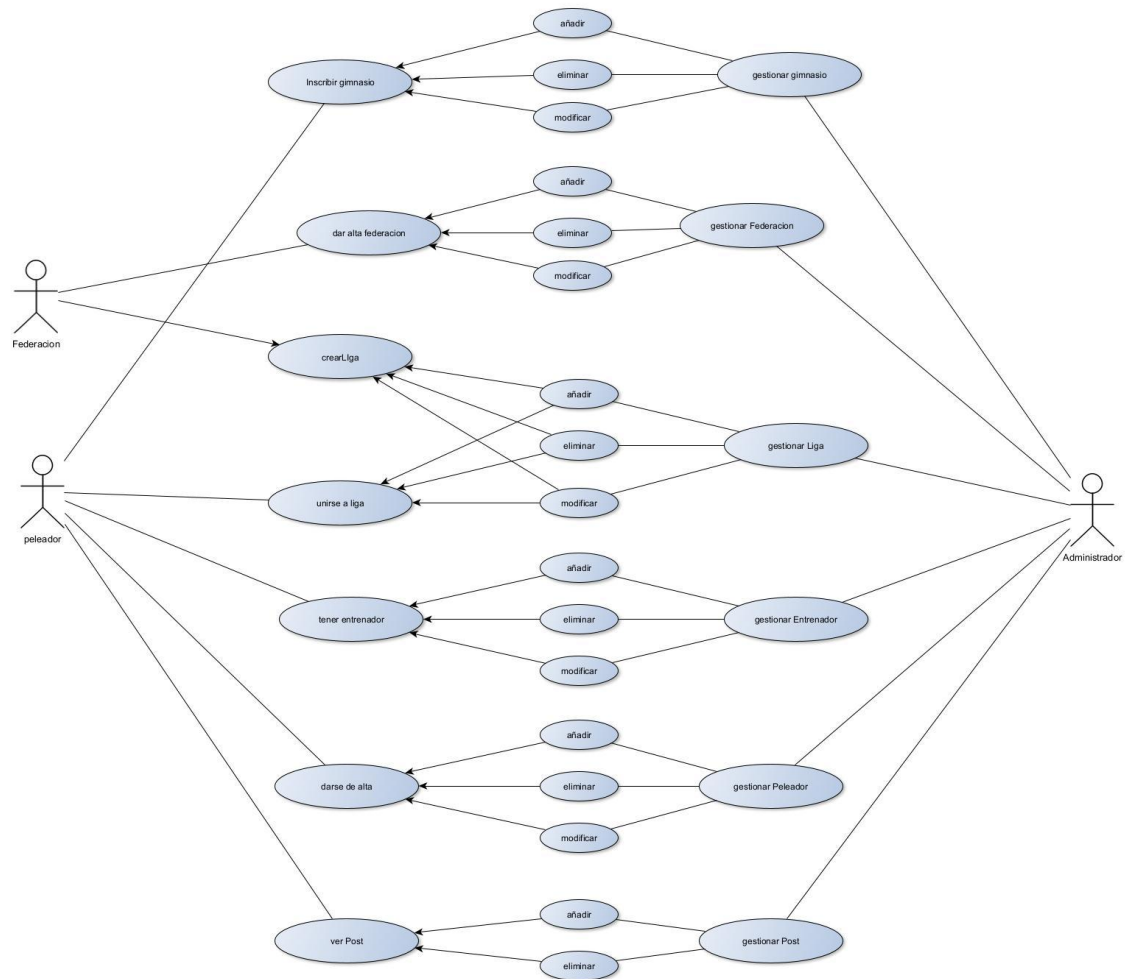
Interfaz de desarrollo= IntelliJ IDEA

Conexión de BBDD= Hibernate

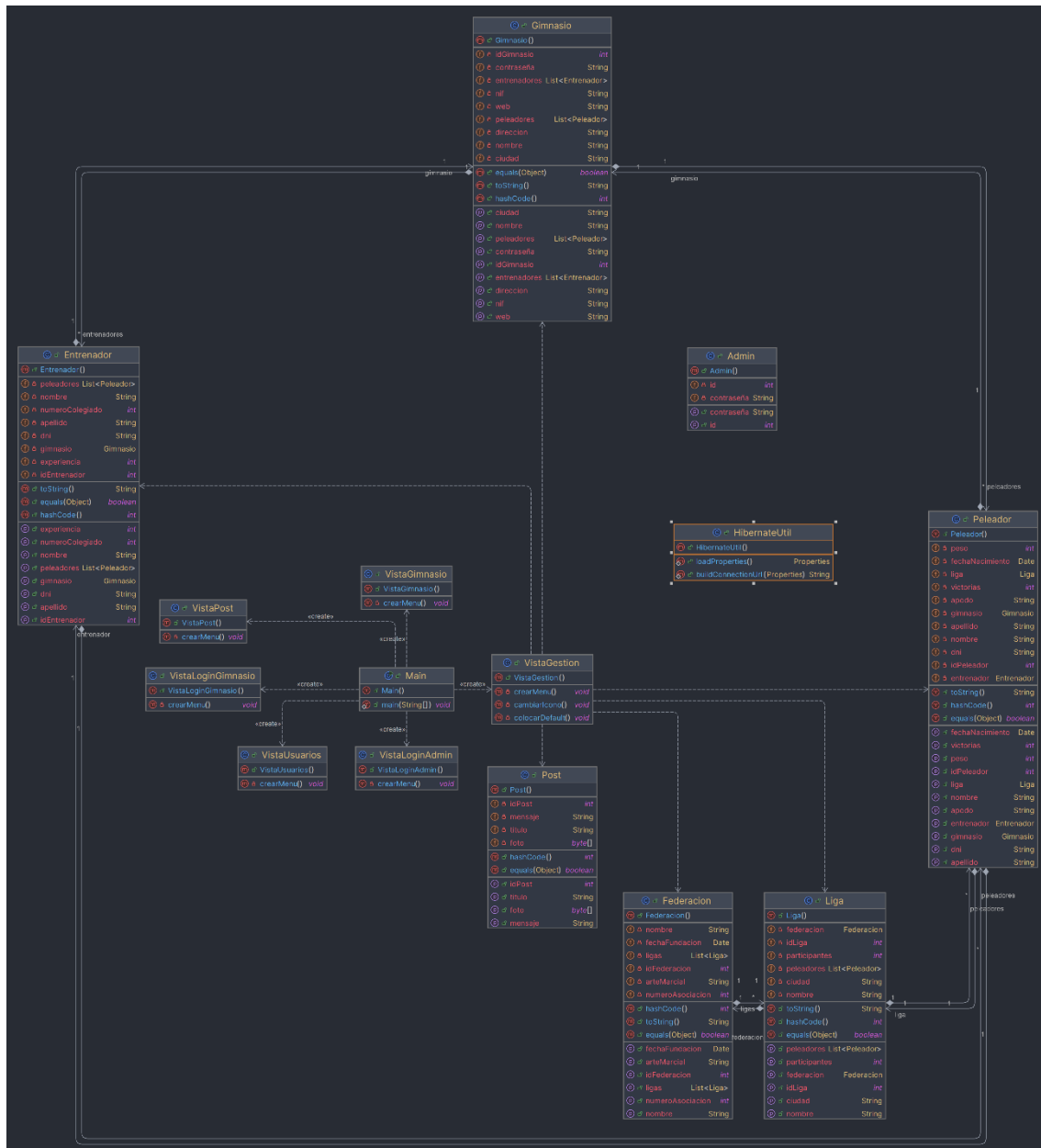
Herramienta de desarrollo de bases de datos= MySQL Workbench

Tipo de bases de datos= MySQL

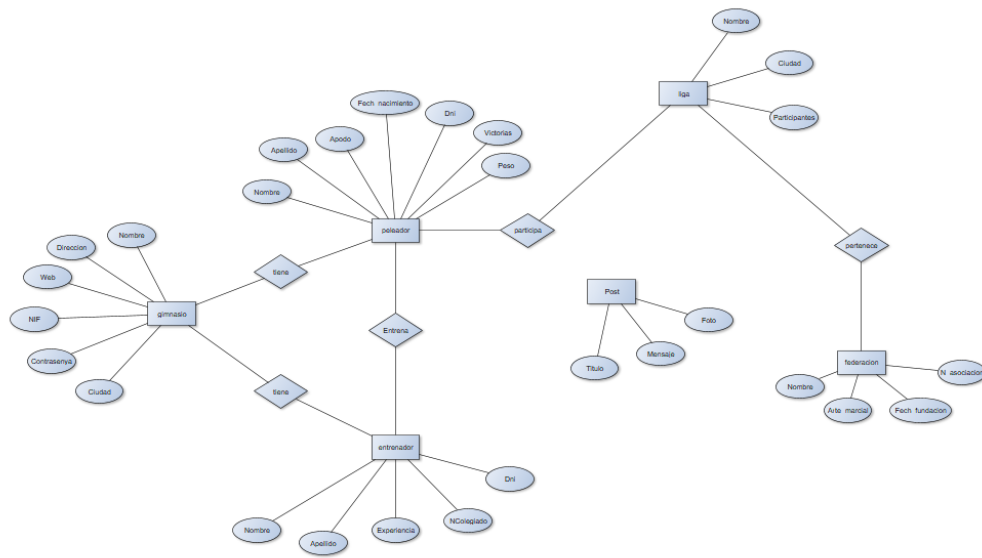
3.2.- Diagrama de casos de uso



3.3.- Diagrama de clases



3.4.- MER



4. Desarrollo de la aplicación

4.1.- Control de usuarios

Tengo 3 tipos de usuarios:

- **Administrador**
- **Gimnasio**
- **Invitado**

Este es el código que utilizo para controlar los inicios de sesión en los diferentes usuarios

```
case "administrador":
    vistaUsers.setVisible(false);
    vistaLoginAdmin.setVisible(true);
    break;
case "gimnasio":

    vistaUsers.setVisible(false);
    comboGimnasioLogin();
    vistaLoginGimnasio.setVisible(true);

    break;
case "invitado":
    vistaUsers.setVisible(false);
    vistaPost.setVisible(true);
    inicializarPost(modelo.getPosts());
    break;
case "login admin":
    String hashGimnasioContraseñaLoginAdmin;
    hashGimnasioContraseñaLoginAdmin =
DigestUtils.sha256Hex(vistaLoginAdmin.administradorContraseñaTxt.getText());
    Admin admin=modelo.getAdmin();
    if
(modelo.verificarContrasena(admin,hashGimnasioContraseñaLoginAdmin)) {
        vistaLoginAdmin.setVisible(false);
        actualizarListas();
        actualizarCombos();
        vistaGestion.pack();
        vistaGestion.setVisible(true);

    }else {
        JOptionPane.showMessageDialog(vistaLoginAdmin, "Contraseña
incorrecta", "Error", JOptionPane.ERROR_MESSAGE);
    }
    break;
case "login gimnasio":
```

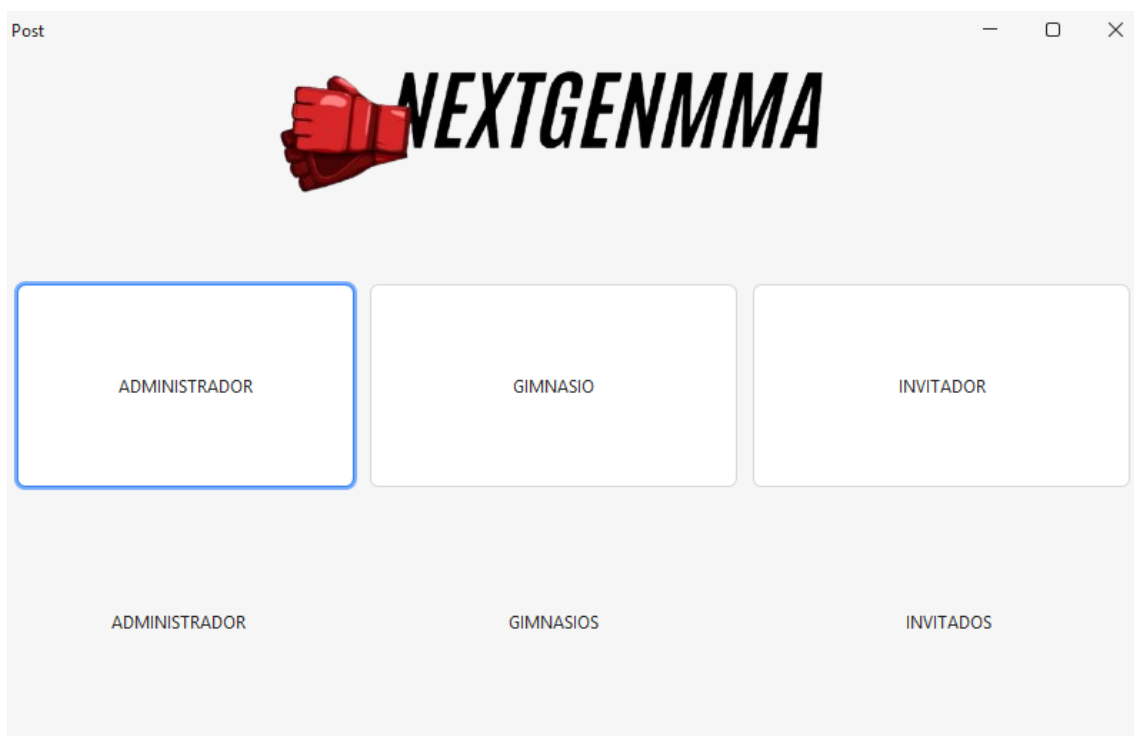
```
String hashGimnasioContraseñaLogin;
hashGimnasioContraseñaLogin =
DigestUtils.sha256Hex(vistaLoginGimnasio.loginGimnasioContraseñaTxt.ge
tText());
if (modelo.verificarContrasenaGimnasio((Gimnasio)
vistaLoginGimnasio.loginGimnasioCombo.getSelectedItem(),
hashGimnasioContraseñaLogin)) {
    vistaLoginGimnasio.setVisible(false);

    vistaGimnasio.setVisible(true);

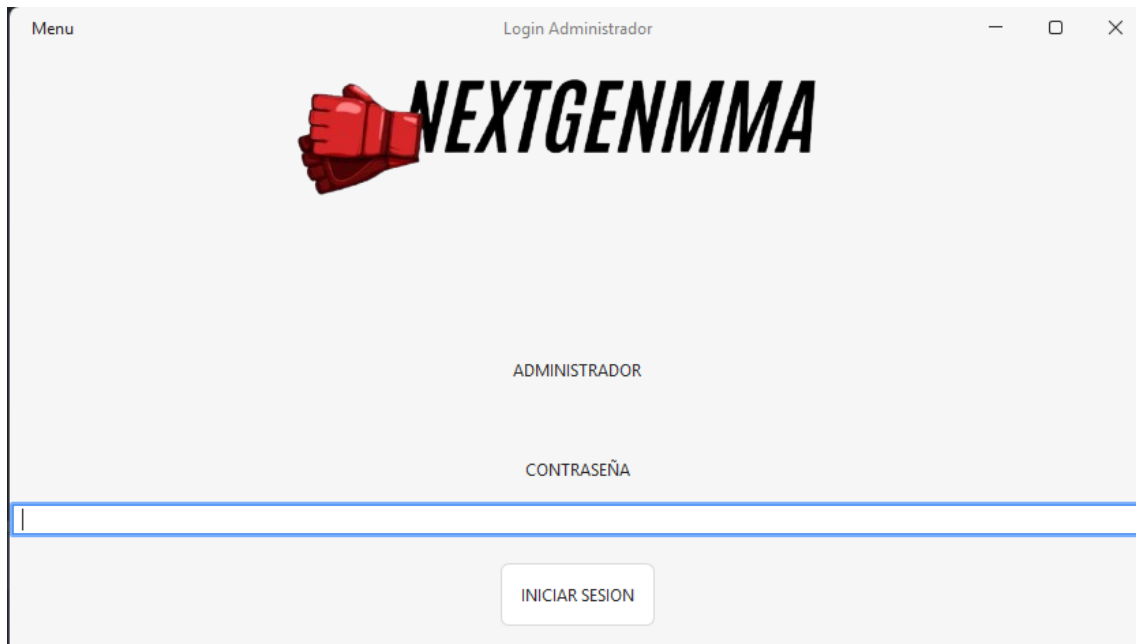
    vistaGimnasio.solicitudPeleadoresActivosTxt.setText(String.valueOf(mod
elo.peleadoresActivos((Gimnasio)
vistaLoginGimnasio.loginGimnasioCombo.getSelectedItem())));

} else {
    JOptionPane.showMessageDialog(vistaLoginGimnasio, "Contraseña
incorrecta", "Error", JOptionPane.ERROR_MESSAGE);
}

break;
```



ADMINISTRADOR



The screenshot shows a web browser window titled "Login Administrador". The page features the "NEXTGENMMA" logo, which consists of a red boxing glove icon and the text "NEXTGENMMA". Below the logo, there are two input fields: "ADMINISTRADOR" and "CONTRASEÑA". The "CONTRASEÑA" field is currently empty and has a blue border. Below the input fields is a button labeled "INICIAR SESION".

Contara con un login tradicional con usuario fijo, el administrador introducirá la contraseña y accederá a la pestaña de Gestion si la contraseña o de lo contrario le saldrá un mensaje de error.

(CONTRASEÑA ERRONEA)



The screenshot shows the same "Login Administrador" page as before, but with an error message displayed. The error message is in a white box with a red exclamation mark icon and the text "Contraseña incorrecta". Below the error message is a blue button labeled "Aceptar". The "CONTRASEÑA" input field is now filled with asterisks (*****). The "INICIAR SESION" button is still visible at the bottom.

(CONTRASEÑA CORRECTA)

PESTAÑA PELEADOR

Menu

PELEADOR GIMNASIO ENTRENADOR LIGA FEDERACION POST INFORMES

NOMBRE

APELLIDO

DNI

PESO

VICTORIAS

APODO

NACIMIENTO

...

GIMNASIO

Gimnasioa', nif='a', direccion='a', ciudad='a

ENTRENADOR

Entrenador{a', apellido='a', dni='22', numeroColegiado=22

LIGA

Liga{nombre='a', ciudad='a', participantes=22

INSERTAR

Peleadora2', apellido='a2', dni='a222', peso=22

Peleadorfsa', apellido='fas', dni='fsa', peso=21

Peleadora23', apellido='a2', dni='a222', peso=22

Peleadora299', apellido='a2', dni='a222', peso=22

MODIFICAR

ELIMINAR

Aquí el administrador podrá eliminar, insertar o modificar los peleadores.

PESTAÑA GIMNASIO

Menu — ×

PELEADOR GIMNASIO ENTRENADOR LIGA FEDERACION POST INFORMES

NOMBRE

DIRECCION

NIF

CIUDAD

CONTRASEÑA

WEB

INSERTAR

MODIFICAR

ELIMINAR

Gimnasioa', nif='a', direccion='a', ciudad='a'
Gimnasioaa', nif='222', direccion='aaa', ciudad='aaa'
Gimnasio333', nif='333', direccion='333', ciudad='333'

Aquí el administrador podrá eliminar, insertar o modificar los Gimnasios
La contraseña se hashea automáticamente.

PESTAÑA ENTRENADOR

Menu — ×

PELEADOR GIMNASIO **ENTRENADOR** LIGA FEDERACION POST INFORMES

NOMBRE

APELLIDO

DNI

NCOLEGIADO

EXPERIENCIA

GIMNASIO

Gimnasioa', nif='a', direccion='a', ciudad='a' ↻

INSERTAR

Entrenador{a', apellido='a', dni='22', numeroColegiado=22
Entrenador{a232', apellido='a2', dni='22', numeroColegiado=22

MODIFICAR

ELIMINAR

Aquí el administrador podrá eliminar, insertar o modificar los Entrenadores.

PESTAÑA LIGA

Menu — ×

PELEADOR GIMNASIO ENTRENADOR LIGA FEDERACION POST INFORMES

NOMBRE

CIUDAD

PARTICIPANTES

FEDERACION

Federacion{a', arteMarcial='a', numeroAsociacion=2222} ↺

INSERTAR

Liga{nombre='a', ciudad='a', participantes=22
Liga{nombre='asda', ciudad='2asd', participantes=22
Liga{nombre='asda2', ciudad='2asd', participantes=22

MODIFICAR

ELIMINAR

Aquí el administrador podrá eliminar, insertar o modificar las Ligas.

PESTAÑA FEDERACION

Menu — ×

PELEADOR GIMNASIO ENTRENADOR LIGA **FEDERACION** POST INFORMES

NOMBRE

Nº ASOCIACION

ARTE MARCIAL

FUNDACION

...

INSERTAR

Federacion{a', arteMarcial='a', numeroAsociacion=2222}
Federacion{a213123', arteMarcial='a', numeroAsociacion=2222}
Federacion{a2131231111', arteMarcial='a', numeroAsociacion=321312}

MODIFICAR

ELIMINAR

Aquí el administrador podrá eliminar, insertar o modificar las Federaciones.

PESTAÑA POST

Menu — ×

PELEADOR GIMNASIO ENTRENADOR LIGA FEDERACION **POST** INFORMES

TITULO

MENSAJE

FOTO

ADJUNTAR

INSERTAR

MODIFICAR

ELIMINAR

diegogil.com.clases.Post@fc74eabd

Aquí el administrador podrá eliminar, insertar o modificar los POST.

PESTAÑA INFORMES

Menu

PELEADOR GIMNASIO ENTRENADOR LIGA FEDERACION POST **INFORMES**

GIMNASIOS	PELEADORES	ENTRENADORES
GENERAR	GENERAR	GENERAR
LIGAS	PELEADORES POR LIGA	PELEADORES POR GIMNASIO
Liga(nombre='a', ciudad='a', participante...		Gimnasioa', nif='a', direccion='a', ciudad...
GENERAR	GENERAR	GENERAR

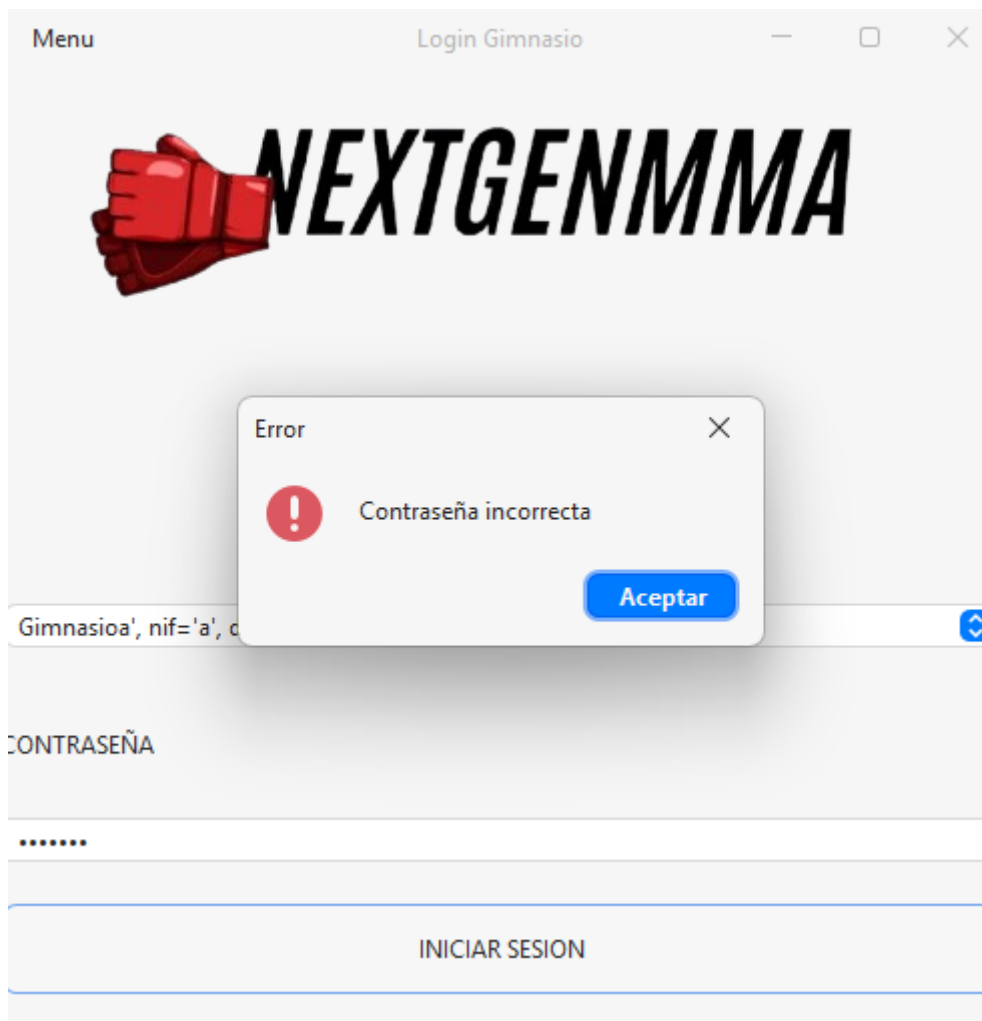


USUARIO GIMNASIO

Login

Contara con un login tradicional con varios usuarios, se seleccionara el gimnasio, este introducirá la contraseña y accederá a la pestaña de características de gimnasio si la contraseña o de lo contrario le saldrá un mensaje de error.

(CONTRASEÑA INCORRECTA)



The screenshot displays a web application window titled "Login Gimnasio". The header features a "Menu" button and the "NEXTGENMMA" logo, which includes a red boxing glove icon. A modal error dialog is centered on the screen, titled "Error" with a close button (X). It contains a red exclamation mark icon and the text "Contraseña incorrecta", followed by a blue "Aceptar" button. Below the dialog, a search bar contains the text "Gimnasioa', nif='a', c". The login form includes a "CONTRASEÑA" label, a password input field with masked characters "*****", and a large "INICIAR SESION" button at the bottom.

(CONTRASEÑA CORRECTA)

PANEL DE SOLICITUD DE CORREO ELECTRONICO

Menu

Gimnasios

SOLICITUD

INFORME

ASUNTO	
NOMBRE	
APELLIDO	
DNI	
PESO	
VICTORIAS	
APODO	
NACIMIENTO	...
GIMNASIO	Gimnasio{a', nif='a', direccion='a', ciudad='a' 
ENTRENADOR	Entrenador{a', apellido='a', dni='22', numeroColegiado=22 
LIGA	Liga{nombre='a', ciudad='a', participantes=22 
ENVIAR	

Aquí el Gimnasio podrá enviar un correo electrónico para realizar acciones sobre sus peleadores

PANEL DE INFORMES

Menu

Gimnasios

SOLICITUD

INFORME

PELEADORES

GENERAR

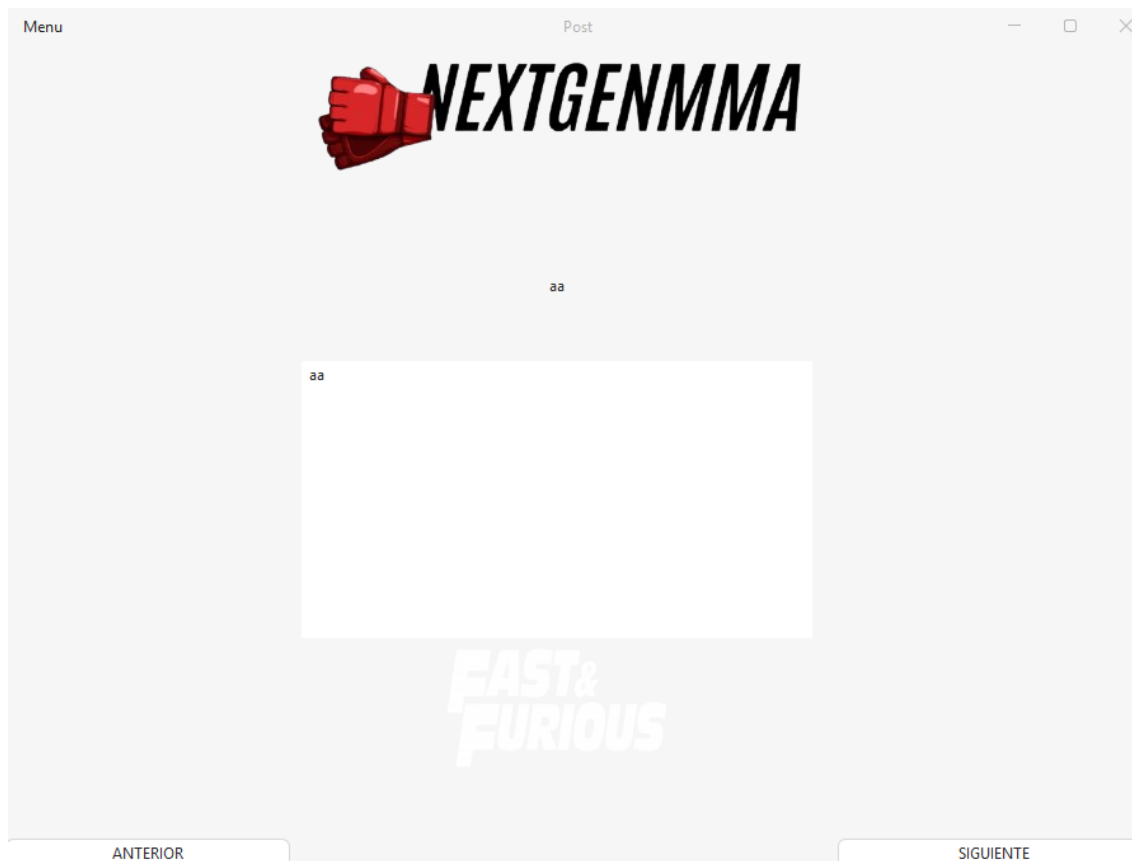
PELEADORES EN ACTIVO

4

En la pestaña de informes el Gimnasio será capaz de generar un informe de todos los peleadores que están en su gimnasio además de ver los peleadores que posee.

INVITADO

PANEL DE INVITADOS/POST



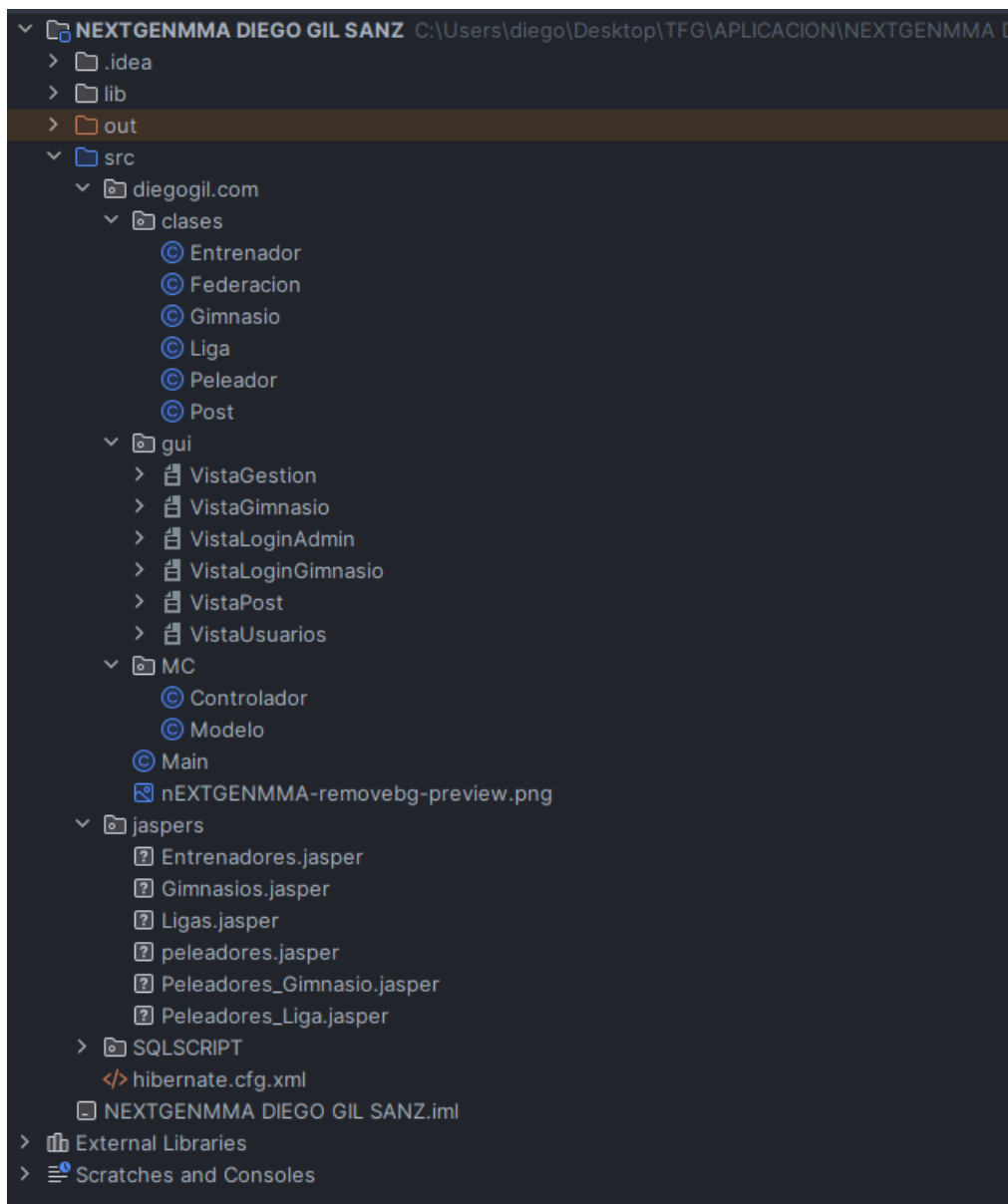
En esta ventana, los invitados podrán ver todos los POST que la publicación ha publicado ordenados del mas nuevo al mas viejo.

4.2.- Desarrollo e implementación de la aplicación

La Estructura de Proyecto cuenta con un total de 6 clases mapeadas, 6 Forms con sus respectivas clases para conformar la interfaz, 1 clase modelo donde alberga todo el código de inserción en la base de datos y funciones de generación de informes además de la conexión y la función de enviar correos electrónicos.

Cuento con 6 jasperreport 2 de ellos les paso parámetros a través de código y se encargan de mostrar información.

Por ultimo y de los mas importante, el main, donde llamo a las clases para que funcione correctamente



MAPEOS

PELEADOR

```
@Entity
public class Peleador {
    5 usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Id
    @Column(name = "id_peleador")
    private int idPeleador;
    6 usages
    @Basic
    @Column(name = "nombre")
    private String nombre;
    6 usages
    @Basic
    @Column(name = "apellido")
    private String apellido;
    6 usages
    @Basic
    @Column(name = "dni")
    private String dni;
    6 usages
    @Basic
    @Column(name = "peso")
    private int peso;
    5 usages
    @Basic
    @Column(name = "victorias")
    private int victorias;
    5 usages
    @Basic
    @Column(name = "apodo")
    private String apodo;
    5 usages
    @Basic
    @Column(name = "fecha_nacimiento")
    private Date fechaNacimiento;
    2 usages
    @ManyToOne
    @JoinColumn(name = "id_entrenador", referencedColumnName = "id_entrenador")
    private Entrenador entrenador;
    2 usages
    @ManyToOne
    @JoinColumn(name = "id_gimnasio", referencedColumnName = "id_gimnasio")
    private Gimnasio gimnasio;
    2 usages
    @ManyToOne
    @JoinColumn(name = "id_liga", referencedColumnName = "id_liga")
    private Liga liga;
```

- **Private int idPeleador:**
El id del peleador es el identificador único por el cual identificaremos a un peleador en la BBDD, se almacena como numero entero, este no lo tendremos que añadir manualmente debido a que es un campo autoIncrement.
- **Private String nombre:**
Es el nombre completo del peleador, se almacena como una cadena de caracteres.
- **Private String apellido:**
Aquí almacenaremos los apellidos del peleador, se almacenaran como una cadena de caracteres.
- **Private String dni:**
En esta columna almacenaremos el documento de identidad del peleador, se admiten NIF, NIE y visados pero el nombre es dni ya que al principio la aplicación esta pensada para empezar en España y lo que mas hay es DNI, se almacena como cadena de caracteres.
- **Private int peso:**
En el campo de peso se almacenara el peso redondeado como numero entero y en formato de kilogramos, la aplicación esta pensada para gestionar la federación y no necesitamos pesos precisos a diferencia de las ligas, este se almacenara como entero en la BBDD.
- **Private int victorias:**
En el campo de victorias se almacenara el numero de victorias del peleador como numero entero.
- **Private String apodo:**
En el campo apodo se almacena el apodo del peleador como cadena de String, como carácter general es el nombre que recibirá en la presentación.
- **Private Date fecha_nacimiento:**
En el campo de fecha de nacimiento se almacenara la fecha seleccionada en formato de YYYY-MM-DD.
- Un peleador puede tener solo un entrenador
- Un peleador solo puede estar en un gimnasio
- Un peleador solo puede estar en una liga.

Entrenador

```
@Entity
public class Entrenador {
    5 usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Id
    @Column(name = "id_entrenador")
    private int idEntrenador;
    6 usages
    @Basic
    @Column(name = "nombre")
    private String nombre;
    6 usages
    @Basic
    @Column(name = "apellido")
    private String apellido;
    6 usages
    @Basic
    @Column(name = "dni")
    private String dni;
    5 usages
    @Basic
    @Column(name = "experiencia")
    private int experiencia;
    6 usages
    @Basic
    @Column(name = "numero_colegiado")
    private int numeroColegiado;
    2 usages
    @OneToMany(mappedBy = "entrenador")
    private List<Peleador> peleadores;
    2 usages
    @ManyToOne
    @JoinColumn(name = "id_gimnasio", referencedColumnName = "id_gimnasio")
    private Gimnasio gimnasio;
```

- **Private int id_entrenador:**
El id del entrenador es el identificador único por el cual identificaremos a un entrenador en la BBDD, se almacena como numero entero, este no lo tendremos que añadir manualmente debido a que es un campo autoIncrement.
- **Private String nombre:**
Es el nombre completo del entrenador, se almacena como una cadena de caracteres.
- **Private String apellido:**
Aquí almacenaremos los apellidos del entrenador, se almacenaran como una cadena de caracteres.
- **Private String dni:**
En esta columna almacenaremos el documento de identidad del entrenador, se admiten NIF, NIE y visados pero el nombre es dni ya que al principio la aplicación esta pensada para empezar en España y lo que mas hay es DNI, se almacena como cadena de caracteres.

- **Private int experiencia**
En este campo almacenamos los años de experiencia como numero entero.
- **Private int numero_colegiado**
Almacenaremos el numero de colegiado como numero entero ya que son solo números
- Un entrenador puede tener a su cargo varios peleadores pero un peleador solo un entrenador
- Un Entrenador pertenece a un gimnasio pero un gimnasio puede tener varios entrenadores

Gimnasio

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Id
@Column(name = "id_gimnasio")
private int idGimnasio;
6 usages
@Basic
@Column(name = "nombre")
private String nombre;
5 usages
@Basic
@Column(name = "contraseña")
private String contraseña;
6 usages
@Basic
@Column(name = "NIF")
private String nif;
6 usages
@Basic
@Column(name = "direccion")
private String direccion;
5 usages
@Basic
@Column(name = "web")
private String web;
6 usages
@Basic
@Column(name = "ciudad")
private String ciudad;
2 usages
@OneToMany(mappedBy = "gimnasio")
private List<Peleador> peleadores;
2 usages
@OneToMany(mappedBy = "gimnasio")
private List<Entrenador> entrenadores;
```

- **private int idGimnasio;**
El id del gimnasio es el identificador único por el cual identificaremos a un gimnasio en la BBDD, se almacena como numero entero, este no lo tendremos que añadir manualmente debido a que es un campo autoIncrement.
- **private String nombre;**
Aqui almacenaremos el nombre del gimnasio como una cadena de caracteres
- **private String contraseña;**
Guardaremos la contraseña del gimnasio hasheada en la BBDD para aumentar la seguridad.
- **private String nif;**
En este campo guardaremos el NIF del gimnasio como cadena de caracteres
- **private String direccion;**
La dirección del gimnasio se almacenara como una cadena de caracteres en la BBDD
- **private String web;**
La pagina web se almacenara como una cadena de String
- **private String ciudad;**
La ciudad se almacenara como una cadena de String
- Un gimnasio puede tener varios peleadores pero un peleador un solo gimnasio
- Un gimnasio puede tener varios entrenadores pero un entrenador un solo gimnasio

Liga

```
@Id
@Column(name = "id_liga")
private int idLiga;
6 usages
@Basic
@Column(name = "nombre")
private String nombre;
6 usages
@Basic
@Column(name = "ciudad")
private String ciudad;
6 usages
@Basic
@Column(name = "participantes")
private int participantes;
2 usages
@ManyToOne
@JoinColumn(name = "id_federacion", referencedColumnName = "id_federacion")
private Federacion federacion;
2 usages
@OneToMany(mappedBy = "liga")
private List<Peleador> peleadores;
```

- **private int idLiga;**
El id de la liga es el identificador único por el cual identificaremos a una liga en la BBDD, se almacena como numero entero, este no lo tendremos que añadir manualmente debido a que es un campo autoIncrement.
- **private String nombre;**
Es el nombre completo de la liga, se almacena como una cadena de caracteres.
- **private String ciudad;**
La ciudad se guardara en la BBDD como una cadena de caracteres
- **private int participantes;**
La cantidad de caracteres se guardaran como un numero entero
- Una liga puede tener varios peleadores pero un peleador una liga
- Una liga pertenece a una federación/promotora y una federación/promotora puede tener varias ligas.

Federacion

```
@GeneratedValue(strategy = GenerationType.IDENTITY)
@Id
@Column(name = "id_federacion")
private int idFederacion;
6 usages
@Basic
@Column(name = "nombre")
private String nombre;
6 usages
@Basic
@Column(name = "arte_marcial")
private String arteMarcial;
5 usages
@Basic
@Column(name = "fecha_fundacion")
private Date fechaFundacion;
6 usages
@Basic
@Column(name = "numero_asociacion")
private int numeroAsociacion;
2 usages
@OneToMany(mappedBy = "federacion")
private List<Liga> ligas;
```

- **private int idFederacion;**
El id de la federacion es el identificador único por el cual identificaremos a una federacion en la BBDD, se almacena como numero entero, este no lo tendremos que añadir manualmente debido a que es un campo autoIncrement.
- **private String nombre;**
Es el nombre completo de la federacion, se almacena como una cadena de caracteres.
- **private String arteMarcial;**
En este campo se almacenara el arte marcial con la que trabaja la federación/promotora
- **private Date fechaFundacion;**
La fecha de fundación se almacenara con formato YYYY/MM/DD
- **private int numeroAsociacion;**
El numero de identificación es la documentación legal que tiene una federación/promotora de identificarse legalmente
- Una federación puede tener varias ligas pero una liga solo una promotora

POST

```
@Id
@Column(name = "id_post")
private int idPost;
5 usages
@Basic
@Column(name = "titulo")
private String titulo;
5 usages
@Basic
@Column(name = "mensaje")
private String mensaje;
5 usages
@Basic
@Column(name = "foto")
private byte[] foto;
```

- **private int idPost;**
El id del post es el identificador único por el cual identificaremos a un post en la BBDD, se almacena como numero entero, este no lo tendremos que añadir manualmente debido a que es un campo autoIncrement.
- **private String titulo;**
En el campo titulo almacenaremos el titulo como una cadena de caracteres
- **private String mensaje;**
El mensaje se almacenara en una cadena de caracteres
- **private byte[] foto;**
La imagen se debe guardar como un array de bytes para su correcto almacenamiento

Conexión

LOCAL

El primer paso es intentar la conexión para comprobar si existe la BBDD, en el caso que exista se desconectara automáticamente y dejara que la conexión mediante hibernate se efectue, en caso de que la BBDD no exista llamara al método encargado de crear la bbdd.

```
public static Connection connect() {
    try {
        if (connection == null || connection.isClosed()) {

            try {
                connection = DriverManager.getConnection("jdbc:" +
                    sqbd + "://" + ip + ":" + port + "/" + name +
                    "?useSSL=false&allowPublicKeyRetrieval=true", user, password);
            } catch (SQLException e) {
                createDatabase();
                try {
                    connection = DriverManager.getConnection("jdbc:" +
                        sqbd + "://" + ip + ":" + port + "/" + name +
                        "?useSSL=false&allowPublicKeyRetrieval=true", user, password);
                } catch (SQLException ex) {
                    JOptionPane.showConfirmDialog(null, "Error al
                    conectar a la base de datos. Asegúrate de que el servidor esté en
                    ejecución y la configuración sea correcta.", "Error",
                    JOptionPane.DEFAULT_OPTION, JOptionPane.ERROR_MESSAGE);
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
    System.out.println("Conectado a la base de datos: " + name + " en
    " + ip + ":" + port + " como " + user + " con contraseña: " +
    password);
    return connection;
}
```

Si la BBDD no esta creada llamara al siguiente método que leera el script de la BBDD adaptado para leerse separando cada sentencia con guiones, una vez creada la BBDD se desconectara y se realizara la conexión mediante hibernate asegurando que puede trabajar en la bbdd

```
private static void createDatabase() {
    try {
        connection = DriverManager.getConnection("jdbc:" + sgbd +
        ":///" + ip + ":" + port +
        "/*useSSL=false&allowPublicKeyRetrieval=true", user, password);
        String code = leerFichero(sqlRoute);
        String[] queries = code.split("--");
        System.out.println("Ejecutando script SQL para crear la base
        de datos...");
        for (String query : queries) {
            query = query.trim();
            if (!query.isEmpty()) {
                try (PreparedStatement statement =
                connection.prepareStatement(query)) {
                    statement.executeUpdate();
                }
            }
        }
    } catch (SQLException e) {
        e.printStackTrace();
    } finally {
        disconnect();
    }
}
```

Una vez nos hemos asegurado de que la BBDD esta creada, conectamos con hibernate

```
public void conectar() {
    try {
        Configuration config = new Configuration();
        config.configure("hibernate.cfg.xml");
        config.addAnnotatedClass(Entrenador.class);
        config.addAnnotatedClass(Gimnasio.class);
        config.addAnnotatedClass(Peleador.class);
        config.addAnnotatedClass(Liga.class);
        config.addAnnotatedClass(Post.class);
        config.addAnnotatedClass(Federacion.class);

        StandardServiceRegistry ssr = new
        StandardServiceRegistryBuilder()
            .applySettings(config.getProperties())
            .build();
        sessionFactory = config.buildSessionFactory(ssr);
    } catch (Exception e) {
        e.printStackTrace();
        System.err.println("Error inicializando Hibernate
        SessionFactory.");
    }
}
```

SERVIDOR

En la versión de servidor al estar la BBDD ya creada no es necesario comprobar que existe y se puede efectuar la conexión directa

```
<property name="connection.url">jdbc:mysql://54.82.86.20:3306/nextgenmma?useSSL=false&allowPublicKeyRetrieval=true</property>
<property name="connection.driver_class">com.mysql.jdbc.Driver</property>
  <property name="connection.username">tfg_api</property>
<property name="hibernate.connection.password">55NN,5&8SH,oMY7F~6v1/t0~;VH[N[G</property>
<property name="hibernate.show_sql">>true</property>
<property name="hibernate.format_sql">>true</property>
  <mapping class="diegogil.com.clases.Entrenador"/>
  <mapping class="diegogil.com.clases.Federacion"/>
  <mapping class="diegogil.com.clases.Gimnasio"/>
  <mapping class="diegogil.com.clases.Liga"/>
  <mapping class="diegogil.com.clases.Peleador"/>
  <mapping class="diegogil.com.clases.Post"/>
```

```
public void conectar() {
    try {
        Configuration config = new Configuration();
        config.configure(resource: "hibernate.cfg.xml");
        config.addAnnotatedClass(Entrenador.class);
        config.addAnnotatedClass(Gimnasio.class);
        config.addAnnotatedClass(Peleador.class);
        config.addAnnotatedClass(Liga.class);
        config.addAnnotatedClass(Post.class);
        config.addAnnotatedClass(Federacion.class);

        StandardServiceRegistry ssr = new StandardServiceRegistryBuilder()
            .applySettings(config.getProperties())
            .build();
        sessionFactory = config.buildSessionFactory(ssr);
    } catch (Exception e) {
        e.printStackTrace();
        System.err.println("Error inicializando Hibernate SessionFactory.");
    }
}
```

INSERCIÓN DE DATOS

A la hora de realizar inserciones, modificaciones o eliminaciones, Creamos una Sesión y realizamos la opción deseada

```
public void altaPeleador(Peleador peleador) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.save(peleador);
        session.getTransaction().commit();
    }
}

1 usage
public void eliminarPeleador(Peleador peleador) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.delete(peleador);
        session.getTransaction().commit();
    }
}

1 usage
public void modificarPeleador(Peleador peleador) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.saveOrUpdate(peleador);
        session.getTransaction().commit();
    }
}
```

```
public void altaEntrenador(Entrenador entrenador) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.save(entrenador);
        session.getTransaction().commit();
    }
}

1 usage
public void eliminarEntrenador(Entrenador entrenador) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.delete(entrenador);
        session.getTransaction().commit();
    }
}

1 usage
public void modificarEntrenador(Entrenador entrenador) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.saveOrUpdate(entrenador);
        session.getTransaction().commit();
    }
}
```

```
1 usage
public void altaGimnasio(Gimnasio gimnasio) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.save(gimnasio);
        session.getTransaction().commit();
    }
}

1 usage
public void eliminarGimnasio(Gimnasio gimnasio) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.delete(gimnasio);
        session.getTransaction().commit();
    }
}

1 usage
public void modificarGimnasio(Gimnasio gimnasio) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.saveOrUpdate(gimnasio);
        session.getTransaction().commit();
    }
}
```

```
1 usage
public void altaLiga(Liga liga) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.save(liga);
        session.getTransaction().commit();
    }
}

1 usage
public void eliminarLiga(Liga liga) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.delete(liga);
        session.getTransaction().commit();
    }
}

1 usage
public void modificarLiga(Liga liga) {
    try (Session session = getSession()) {
        session.beginTransaction();
        session.saveOrUpdate(liga);
        session.getTransaction().commit();
    }
}
```



```
public void altaFederacion(Federacion federacion) {  
    try (Session session = getSession()) {  
        session.beginTransaction();  
        session.save(federacion);  
        session.getTransaction().commit();  
    }  
}  
  
1 usage  
public void eliminarFederacion(Federacion federacion) {  
    try (Session session = getSession()) {  
        session.beginTransaction();  
        session.delete(federacion);  
        session.getTransaction().commit();  
    }  
}  
  
1 usage  
public void modificarFederacion(Federacion federacion) {  
    try (Session session = getSession()) {  
        session.beginTransaction();  
        session.saveOrUpdate(federacion);  
        session.getTransaction().commit();  
    }  
}
```

```
public void altaPost(Post post) {  
    try (Session session = getSession()) {  
        session.beginTransaction();  
        session.save(post);  
        session.getTransaction().commit();  
    }  
}  
  
1 usage  
public void eliminarPost(Post post) {  
    try (Session session = getSession()) {  
        session.beginTransaction();  
        session.delete(post);  
        session.getTransaction().commit();  
    }  
}  
  
1 usage  
public void modificarPost(Post post) {  
    try (Session session = getSession()) {  
        session.beginTransaction();  
        session.saveOrUpdate(post);  
        session.getTransaction().commit();  
    }  
}
```

CONTROLADOR

- **Insertion**

A la hora de insertar cada uno de los elementos, Creamos el objeto, lo rellenamos con los campos que se rellenan en la interfaz grafica.

Este proceso comprueba si hay algun campo vacio u erroneo para corregirlo antes de la insercion para evitar errores, una vez rellenado, le damos al boton de insercion y genera una ventana para verificar la accion, si la respuesta es si, se insertara.

```
case "añadir peleador":
    if (JOptionPane.showConfirmDialog(vistaGestion, message: "¿Estás seguro de que quieres añadir este peleador?", title: "Confirmar acción", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        if (validarCamposPeleador()) {
            Peleador peleador = new Peleador();
            peleador.setNombre(vistaGestion.peleadorNombreTxt.getText());
            peleador.setApellido(vistaGestion.peleadorApellidoTxt.getText());
            peleador.setDni(vistaGestion.peleadorDniTxt.getText());
            peleador.setPeso(Integer.parseInt(vistaGestion.peleadorPesoTxt.getText()));
            peleador.setVictorias(Integer.parseInt(vistaGestion.peleadorVictoriasTxt.getText()));
            peleador.setApodo(vistaGestion.peleadorApodoTxt.getText());
            peleador.setFechaMacimiento(Date.valueOf(vistaGestion.peleadorMacimientoTxt.getDate()));
            peleador.setGimnasio((Gimnasio) vistaGestion.peleadorGimnasioCombo.getSelectedItem());
            peleador.setEntrenador((Entrenador) vistaGestion.peleadorEntrenadorCombo.getSelectedItem());
            peleador.setLiga((Liga) vistaGestion.peleadorLigaCombo.getSelectedItem());
            modelo.altaPeleador(peleador);
            listarPeleadores(modelo.getPeleadores());
        }
    }
    break;
```

- **Modificacion**

A la hora de la modificacion el usuario necesita seleccionar uno de los objetos de la lista, tiene un control de errores para evitar problemas en la ejecucion, el usuario modifica los datos y una vez clicado en modificacion, saldra una ventana emergente que confirmara la accion

```
case "modificar peleador":
    if (JOptionPane.showConfirmDialog(vistaGestion, message: "¿Estás seguro de que quieres modificar este peleador?", title: "Confirmar acción", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        if (validarCamposPeleador()) {
            Peleador peleadorModificar = (Peleador) vistaGestion.peleadorLista.getSelectedValue();
            peleadorModificar.setNombre(vistaGestion.peleadorNombreTxt.getText());
            peleadorModificar.setApellido(vistaGestion.peleadorApellidoTxt.getText());
            peleadorModificar.setDni(vistaGestion.peleadorDniTxt.getText());
            peleadorModificar.setPeso(Integer.parseInt(vistaGestion.peleadorPesoTxt.getText()));
            peleadorModificar.setVictorias(Integer.parseInt(vistaGestion.peleadorVictoriasTxt.getText()));
            peleadorModificar.setApodo(vistaGestion.peleadorApodoTxt.getText());
            peleadorModificar.setGimnasio((Gimnasio) vistaGestion.peleadorGimnasioCombo.getSelectedItem());
            peleadorModificar.setEntrenador((Entrenador) vistaGestion.peleadorEntrenadorCombo.getSelectedItem());
            peleadorModificar.setLiga((Liga) vistaGestion.peleadorLigaCombo.getSelectedItem());
            modelo.modificarPeleador(peleadorModificar);
            listarPeleadores(modelo.getPeleadores());
        }
    }
    break;
```

- **Eliminacion**

A la hora de eliminar una columna, En cada uno hay que seleccionar en la lista y darle a eliminar, saldra una ventana emergente que buscara confirmar la accion, si la respuesta es si, la elimina

```
case "eliminar peleador":
    if (JOptionPane.showConfirmDialog(vistaGestion, message: "¿Estás seguro de que quieres eliminar este peleador?", title: "Confirmar acción", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        try {
            modelo.eliminarPeleador((Peleador) vistaGestion.peleadorLista.getSelectedValue());
            listarPeleadores(modelo.getPeleadores());
        } catch (Exception ex) {
            JOptionPane.showMessageDialog(vistaGestion, message: "No se puede eliminar el peleador porque tiene peleas asociadas o no existen peleadores.", title: "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
    break;
```

Para insertar un post por lo general es necesaria una imagen, el metodo adjuntar imagen, nos permite seleccionar una imagen filtrada gracias a JFileChooser, una vez seleccionada, redimensionamos la imagen para que corresponda con las medidas de la interfaz, la almacenamos como un array de bytes en formato png y estara lista para la insercion

```
case "adjuntar imagen":
    JFileChooser fileChooser = new JFileChooser();
    fileChooser.setFileFilter(new javax.swing.filechooser.FileNameExtensionFilter( "Imágenes", ".extensions: ".jpg", ".png", ".jpeg"));
    int result = fileChooser.showOpenDialog(vistaGestion);
    if (result == JFileChooser.APPROVE_OPTION) {
        java.io.File file = fileChooser.getSelectedFile();
        try {
            BufferedImage imagenOriginal = javax.imageio.ImageIO.read(file);

            Image imagenRedimensionada = imagenOriginal.getScaledInstance( width: 200, height: 200, Image.SCALE_SMOOTH);
            BufferedImage bufferedResizedImage = new BufferedImage( width: 200, height: 200, BufferedImage.TYPE_INT_ARGB);
            Graphics2D g2d = bufferedResizedImage.createGraphics();
            g2d.drawImage(imagenRedimensionada, x: 0, y: 0, observer: null);
            g2d.dispose();

            // Convertir la imagen redimensionada a un array de bytes
            ByteArrayOutputStream baos = new ByteArrayOutputStream();
            javax.imageio.ImageIO.write(bufferedResizedImage, formatName: "png", baos);
            vistaGestion.postImagenSeleccionada = baos.toByteArray(); // Guarda la imagen redimensionada
            System.out.println("Imagen seleccionada y redimensionada: " + file.getName());
        } catch (java.io.IOException ex) {
            JOptionPane.showMessageDialog(vistaGestion, message: "Error al cargar la imagen: " + ex.getMessage(), title: "Error", JOptionPane.ERROR_MESSAGE);
        }
    }
    break;
```

Para insertar un post, creamos el objeto post, lo rellenamos con los campos de la interfaz y la imagen seleccionada, a la hora de insertar, aparecera la ventana de confirmacion y si la respuesta es positiva se insertara

```
case "insertar post":
    if (JOptionPane.showConfirmDialog(vistaGestion, message: "¿Estás seguro de que quieres insertar este post?", title: "Confirmar acción", JOptionPane.YES_NO_OPTION) == JOptionPane.YES_OPTION) {
        if (validarCamposPost()) {
            Post post = new Post();
            post.setTitulo(vistaGestion.postTituloTxt.getText());
            post.setMensaje(vistaGestion.postMensajeTxt.getText());
            post.setFoto(vistaGestion.postImagenSeleccionada); // Asigna la imagen seleccionada
            modelo.altaPost(post);
            listarPosts(modelo.getPosts());
        }
    }
    break;
```

Obtencion de datos

Almacenaremos todos los datos obtenidos en Arraylist según la tabla de la que provengan.

```
9 usages
public ArrayList<Gimnasio> getGimnasios() {
    try (Session session = getSession()) {
        session.beginTransaction();
        ArrayList<Gimnasio> gimnasios = (ArrayList<Gimnasio>) session.createQuery(s: "from Gimnasio").list();
        session.getTransaction().commit();
        return gimnasios;
    }
}

6 usages
public ArrayList<Entrenador> getEntrenadores() {
    try (Session session = getSession()) {
        session.beginTransaction();
        ArrayList<Entrenador> entrenadores = (ArrayList<Entrenador>) session.createQuery(s: "from Entrenador").list();
        session.getTransaction().commit();
        return entrenadores;
    }
}

4 usages
public ArrayList<Peleador> getPeleadores() {
    try (Session session = getSession()) {
        session.beginTransaction();
        ArrayList<Peleador> peleadores = (ArrayList<Peleador>) session.createQuery(s: "from Peleador").list();
        session.getTransaction().commit();
        return peleadores;
    }
}
```

```
public ArrayList<Liga> getLigas() {
    try (Session session = getSession()) {
        session.beginTransaction();
        ArrayList<Liga> ligas = (ArrayList<Liga>) session.createQuery(s: "from Liga").list();
        session.getTransaction().commit();
        return ligas;
    }
}

4 usages
public ArrayList<Federacion> getFederaciones() {
    try (Session session = getSession()) {
        session.beginTransaction();
        ArrayList<Federacion> federaciones = (ArrayList<Federacion>) session.createQuery(s: "from Federacion").list();
        session.getTransaction().commit();
        return federaciones;
    }
}

23 usages
public ArrayList<Post> getPosts() {
    try (Session session = getSession()) {
        session.beginTransaction();
        ArrayList<Post> posts = (ArrayList<Post>) session.createQuery(s: "from Post").list();
        session.getTransaction().commit();
        return posts;
    }
}
```

Envío de correo

Gracias a la librería javax-mail puedo enviar un mensaje a un correo electrónico, creando una sesión y el mensaje de una forma muy sencilla. Solo hay que indicar el emisor, datos de la cuenta emisora, correo electrónico receptor, rellenar el mensaje por apartador y mandarlo con transport.

```
Usage
public void enviarCorreo(Peleador peleador, String asunto) {
    String host = "smtp.gmail.com";
    String puerto = "587";
    String remitente = "diegogilzg@gmail.com";
    String contraseña = "XXXXXXXXXXXX";

    Properties props = new Properties();
    props.put("mail.smtp.auth", "true");
    props.put("mail.smtp.starttls.enable", "true");
    props.put("mail.smtp.host", host);
    props.put("mail.smtp.port", puerto);

    javax.mail.Session session = javax.mail.Session.getInstance(props, getPasswordAuthentication() -> {
        return new PasswordAuthentication(remitente, contraseña);
    });

    Message mensaje = new MimeMessage(session);
    mensaje.setFrom(new InternetAddress(remitente));
    mensaje.setRecipients(Message.RecipientType.TO, InternetAddress.parse(addresslist: "diegogilzg@gmail.com"));
    mensaje.setSubject(asunto);
    mensaje.setContent(contenido, "text/html; charset=utf-8");
}
```

Control de errores

El control de errores es personalizado para cada entidad, teniendo en cuenta posibles errores a la hora de realizar cualquier acción, las siguientes imágenes muestran como controlo cada campo de un peleador, pero también está el control de errores de cada una de las entidades.

Se verifica que cada campo esté correctamente y que las fechas no puedan ser futuras.

```
private boolean validarCamposPeleador() {
    // campos vacios
    if (vistaGestion.peleadorNombreTxt.getText().isEmpty() ||
        vistaGestion.peleadorApellidoTxt.getText().isEmpty() ||
        vistaGestion.peleadorDniTxt.getText().isEmpty() ||
        vistaGestion.peleadorPesoTxt.getText().isEmpty() ||
        vistaGestion.peleadorVictoriasTxt.getText().isEmpty() ||
        vistaGestion.peleadorApodoTxt.getText().isEmpty() ||
        vistaGestion.peleadorNacimientoTxt.getDate() == null ||
        vistaGestion.peleadorGimnasioCombo.getSelectedIndex() == null ||
        vistaGestion.peleadorEntrenadorCombo.getSelectedIndex() == null) {
        JOptionPane.showMessageDialog(vistaGestion, "Todos los campos del peleador son obligatorios.", "Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }

    if (vistaGestion.peleadorNombreTxt.getText().length() > 50) {
        JOptionPane.showMessageDialog(vistaGestion, "El nombre no puede exceder 50 caracteres.", "Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }

    if (vistaGestion.peleadorApellidoTxt.getText().length() > 50) {
        JOptionPane.showMessageDialog(vistaGestion, "El apellido no puede exceder 50 caracteres.", "Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }

    if (vistaGestion.peleadorApodoTxt.getText().length() > 30) {
        JOptionPane.showMessageDialog(vistaGestion, "El apodo no puede exceder 30 caracteres.", "Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }

    if (vistaGestion.peleadorDniTxt.getText().length() > 15) {
        JOptionPane.showMessageDialog(vistaGestion, "El DNI no puede exceder 15 caracteres.", "Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
}
```

```
try {
    int peso = Integer.parseInt(vistaGestion.peleadorPesoTxt.getText());
    if (peso <= 0) {
        JOptionPane.showMessageDialog(vistaGestion, "El peso debe ser mayor a 0.", "Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(vistaGestion, "El peso debe ser un valor numérico.", "Error", JOptionPane.ERROR_MESSAGE);
    return false;
}

try {
    int victorias = Integer.parseInt(vistaGestion.peleadorVictoriasTxt.getText());
    if (victorias < 0) {
        JOptionPane.showMessageDialog(vistaGestion, "El número de victorias no puede ser negativo.", "Error", JOptionPane.ERROR_MESSAGE);
        return false;
    }
} catch (NumberFormatException e) {
    JOptionPane.showMessageDialog(vistaGestion, "El número de victorias debe ser un valor numérico.", "Error", JOptionPane.ERROR_MESSAGE);
    return false;
}

if (LocalDate.parse(vistaGestion.peleadorNacimientoTxt.getDate().toString()).isAfter(LocalDate.now())) {
    JOptionPane.showMessageDialog(vistaGestion, "La fecha de nacimiento no puede ser futura.", "Error", JOptionPane.ERROR_MESSAGE);
    return false;
}

return true;
}
```

5. Pruebas e informes de la aplicación

5.1.- Estrategia de pruebas

Realizar diferentes situaciones y apuntar en un Excel los resultados

Distribuir la aplicación y repartir un formulario a los usuarios para evaluar los resultados

5.2.- Pruebas realizadas

La aplicación lleva a cabo 2 tipos de pruebas.

- Pruebas de usabilidad:
Mediante un cuestionario que recoge feedback del usuario, el cual ha podido probar/ver la aplicación, puedo verificar que la aplicación es sencilla e intuitiva para que el usuario tenga la mejor experiencia
- Pruebas de caja negra(Pruebas negativas):
He creado un Excel donde indica errores ante situaciones donde hay diferentes entradas de datos que están mal o simplemente que no se realiza, simulando posibles situaciones

5.3.- Informes

Datos de la prueba de usabilidad

Enlace al Excel de [DATOS](#)

Como podemos ver he tenido diversos usuarios con diferentes perfiles, se puede apreciar que han respondido mas aficionados que deportistas, pero los deportistas que tienen una visión mas cercana a cerca de las necesidades del peleador se ven muy satisfechos.

Otro factor muy importante ha sido la respuesta de un peleador el cual esta en la liga mas importante de España(**WOW FC**), se ve muy contento ante la aplicación, dice que es completa lo cual es un resultado muy positivo porque el resultado que queremos es que los peleadores sean los usuarios mas beneficiados junto con la federación.



-Datos de la prueba de caja negra

Enlace al Excel de [DATOS](#) de la prueba de cada negra

Aquí podemos ver un caso donde probamos la inserción de un peleador con diversas situaciones, estas, tienen un mensaje personalizado para lograr solucionar los posibles errores que pueda encontrar el usuario.

En este caso solo estoy mostrando un apartado pero de cada entidad hay un apartado, cada uno con sus situaciones de inserción, modificación y eliminación.

 PELEADOR											
CAMPO	ACCION	ACCION	ACCION	ACCION	ACCION	ACCION	ACCION	ACCION	ACCION	ACCION	ACCION
NOMBRE	DATOS CORRECTOS	FORMATO INCORRECTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS
APELLIDO	DATOS CORRECTOS	DATOS CORRECTOS	FORMATO INCORRECTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS
DNI	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	FORMATO INCORRECTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS
PESO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	FORMATO INCORRECTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS
VICTORIAS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	FORMATO INCORRECTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS
APODO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	FORMATO INCORRECTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS
NACIMIENTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	FORMATO INCORRECTO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS
GIMNASIO	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	VACIO	DATOS CORRECTOS	DATOS CORRECTOS
ENTRENADOR	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	VACIO	DATOS CORRECTOS
LIGA	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	DATOS CORRECTOS	VACIO
RESULTADOS BOTONES											
INSERCIÓN	INSERCIÓN CORRECTA	ALERTA NOMBRE	ALERTA APELLIDO	FORMATO DNI INCORRECTO	EL PESO TIENE QUE SER UN VALOR NUMÉRICO	LAS VICTORIAS TIENEN QUE SER UN VALOR NUMÉRICO	ALERTA APODO	ALERTA NACIMIENTO INCORRECTO	ALERTA NO PUEDE ESTAR VACIO	ALERTA NO PUEDE ESTAR VACIO	INSERCIÓN CORRECTA
MODIFICAR	MODIFICACIÓN CORRECTA	ALERTA NOMBRE	ALERTA APELLIDO	FORMATO DNI INCORRECTO	EL PESO TIENE QUE SER UN VALOR NUMÉRICO	LAS VICTORIAS TIENEN QUE SER UN VALOR NUMÉRICO	ALERTA APODO	ALERTA NACIMIENTO INCORRECTO	ALERTA NO PUEDE ESTAR VACIO	ALERTA NO PUEDE ESTAR VACIO	MODIFICACIÓN CORRECTA
ELIMINAR	ELIMINACIÓN CORRECTA										

CASOS EXCEPCIONALES	
CASO	RESULTADO
INTENTO DE MODIFICACION SIN SELECCIONAR PELEADOR	ALERTA, PELEADOR NO SELECCIONADO O NO EXISTE
INTENTO DE ELIMINACION SIN SELECCIONAR PELEADOR	ALERTA, PELEADOR NO SELECCIONADO O NO EXISTE
MÚLTIPLES ERRORES DE FORMATO A LA HORA DE INSERCIÓN O MODIFICACIÓN	LA APLICACIÓN MOSTRARA ALERTAS SEGÚN EL ORDEN DE LOS CAMPOS EN LA INTERFAZ

6. Implantación y documentación

6.1.- Exportación y distribución de la aplicación

Dentro de cada proyecto (Servidor y local) están sus correspondientes jar y exe.

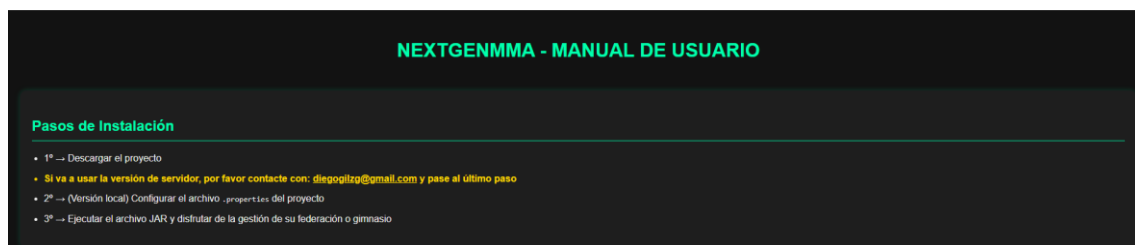
Los JAR son totalmente funcionales al igual que los exe, los cuales han sido creados con la herramienta launch4j.

Para descargar la aplicación, será necesaria la descarga completa del repositorio de github, aquí se encontraran las 2 versiones para su uso..

6.2.- Manual de instalación

[Archivo de Manual de instalación](#)

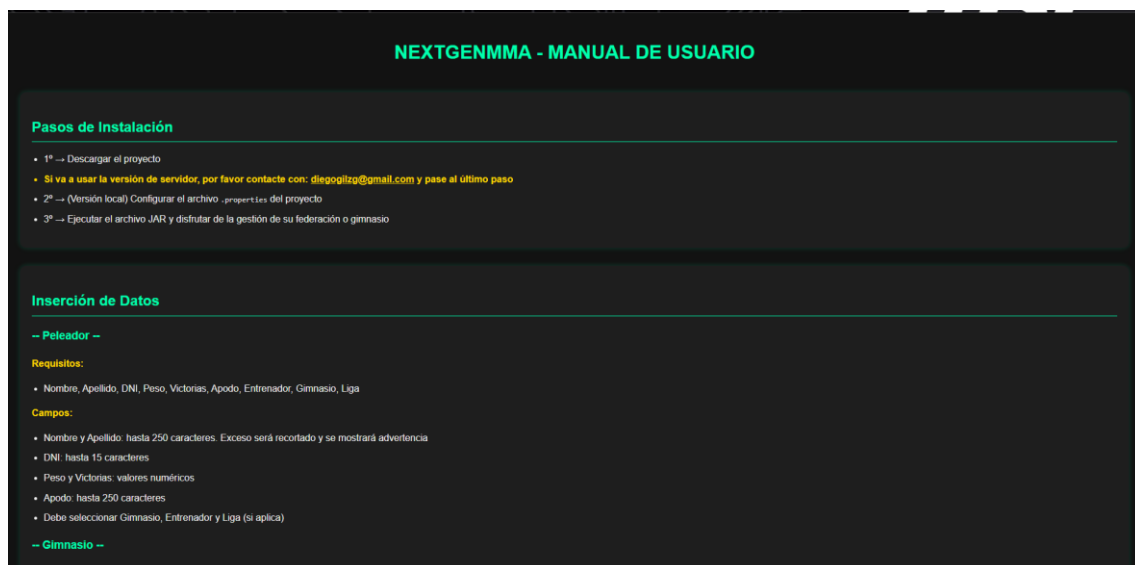
El manual de usuarios que he creado es muy sencillo e intuitivo, doy la opción de instalación para las 2 versiones, para que sea una experiencia simple y satisfactoria



6.3.- Manual de usuario

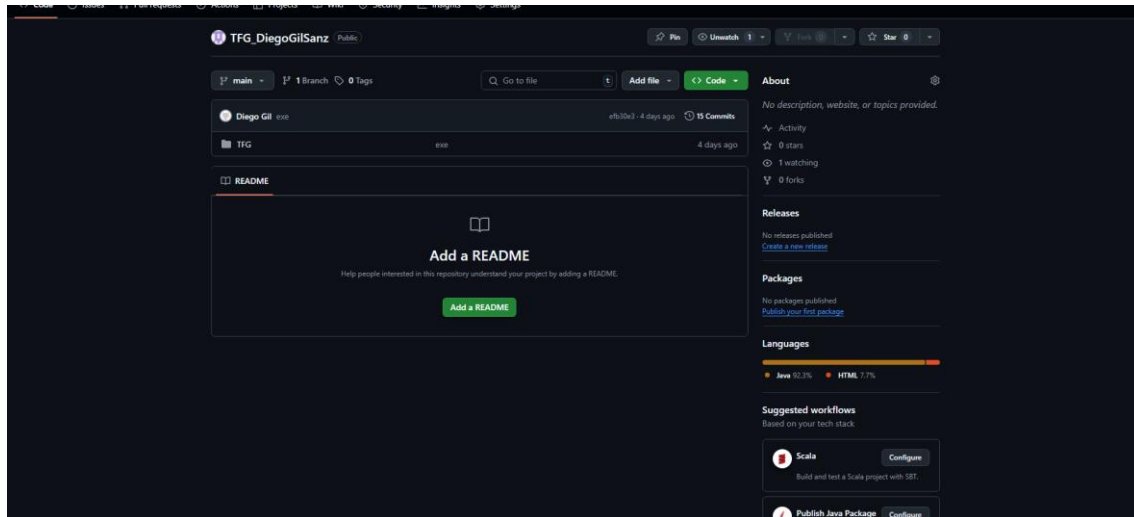
[Archivo de manual de Usuario](#)

El manual de Usuario muestra instrucciones claras para rellenar campos y evitar errores, este es un pequeño fragmento pero en el hipervínculo que puedes ver encima, podrás ver toda la información que contiene



6.4.- Repositorio de la aplicación

https://github.com/DiegoGilSanz/TFG_DiegoGilSanz.git



Control de versiones

exe	DiegoGilSanz pushed 1 commit to main • b574564...efb30e3 • 3 days ago	...
igualar	DiegoGilSanz pushed 1 commit to main • c0d9eac...b574564 • 4 days ago	...
completo	DiegoGilSanz pushed 1 commit to main • 28a2ed4...c0d9eac • 5 days ago	...
Reads Manuales	DiegoGilSanz pushed 1 commit to main • 3b53db7...28a2ed4 • 6 days ago	...
graficos	DiegoGilSanz pushed 1 commit to main • 76fb7af...3b53db7 • 8 days ago	...
server 2	DiegoGilSanz pushed 1 commit to main • 8e83e2d...76fb7af • 14 days ago	...
completado server	DiegoGilSanz force pushed to main • de4ace5...8e83e2d • 14 days ago	...
correcciones oportunas	DiegoGilSanz pushed 1 commit to main • 7058ec0...de4ace5 • 17 days ago	...
Preparacion revision	DiegoGilSanz pushed 1 commit to main • 98f50ec...7058ec0 • 17 days ago	...
completado4	DiegoGilSanz pushed 1 commit to main • 98ba716...98f50ec • 17 days ago	...
completado2	DiegoGilSanz pushed 1 commit to main • f30626...98ba716 • 18 days ago	...
version final 1	DiegoGilSanz pushed 1 commit to main • 07949c2...f30626 • 20 days ago	...
commit seguridad	DiegoGilSanz pushed 1 commit to main • 9171461...07949c2 • 25 days ago	...
Segunda basica opciones medias	DiegoGilSanz pushed 1 commit to main • 6a30b8f...9171461 • on 21 abr	...
primera version funciones basicas	DiegoGilSanz pushed 1 commit to main • d15dfaf...6a30b8f • on 19 abr	...
primera version sin terminar	DiegoGilSanz created main • d15dfaf • on 19 abr	...

6.5.- JavaDoc

[Enlace directo a javadoc](#)

All Classes

Packages

diegojl.com

diegojl.com.classes

diegojl.com.config

diegojl.com.gui

diegojl.com.mc

All Classes

Admin

Controlador

Entrenador

Federacion

Gimnasio

HibernateUtil

Liga

Main

Muchos

Paleador

Post

VistaDecision

VistaGimnasio

VistaJugadorMas

VistaJugadorOmnibus

VistaPost

VistaUsuarios

OVERVIEW

PACKAGE

CLASS

TREE

DEPRECATED

INDEX

HELP

PREV

NEXT

FRAMES

NO FRAMES

Packages

Package	Description
diegojl.com	
diegojl.com.classes	
diegojl.com.config	
diegojl.com.gui	
diegojl.com.mc	

7.- Bibliografía

- Librería hash
https://commons.apache.org/proper/commons-codec/download_codec.cgi
- Hibernate
<https://hibernate.org/orm/documentation/7.0/>
- Aibin, Michal. "Guía Javax-mail." Baeldung, 20 August 2024,
<https://www.baeldung.com/java-email>. Accessed 25 May 2025
- "Dependencias JasperReport." *Dependencias JasperReport*,
https://drive.google.com/file/d/1_79cmdY5ldZmrSL8-LuW7QVWrb6jv_l9/view
- "Descarga de Launch4j." SourceForge,
<https://sourceforge.net/projects/launch4j/files/launch4j-3/3.50/>
- "Descarga JasperReport." *JasperReport*,
<https://sourceforge.net/projects/jasperreports/files/archive/jasperreports/>
- "GitJavaMail." *Java EE*, 28 April 2017, <https://javaee.github.io/javamail/>
- "Librería Flatlaf." *Flatlaf*, Desconocido,
<https://repo1.maven.org/maven2/com/formdev/flatlaf/3.5.2/flatlaf-3.5.2.jar>
- "Librería hash." *Apache Commons*, 24 January 2025,
https://commons.apache.org/proper/commons-codec/download_codec.cgi
- "MySQL Workbench." *MySQL*, <https://www.mysql.com/products/workbench/>
- "yEd - Graph Editor." *yWorks*, <https://www.yworks.com/products/yed>
- "IntelliJ IDEA – the IDE for Pro Java and Kotlin Development." *JetBrains*,
<https://www.jetbrains.com/idea/>
- ChatGPT
<https://chatgpt.com/>

8.- Anexos

Carpeta de proyecto

Diagrama de clases

Diagrama de casos de uso

Script BBDD

Modelo entidad relacion

Parte de ingles

Plan de empresa

Prueba de usabilidad(Caja negra)

Resultado del formulario a los usuarios

Ayuda hacia el usuario

Manual de usuario